

Module: Introduction aux Systèmes Embarqués

Chapitre 01

Introduction aux systèmes embarqués



Plan du cours

Définition et généralités

Conception des systèmes embarqués

Conception matérielle

Conception logicielle

Conception Mixte

Résumé

Plan du cours

Définition et généralités

Conception des systèmes embarqués

Conception matérielle

Conception logicielle

Conception Mixte

Résumé

Système embarqué

- ❑ En observant notre environnement quotidien, on peut facilement constater la présence de systèmes embarqués.
 - ❑ Par exemple, le radioréveil qui vous réveille chaque matin.
 - ❑ Vous programmez votre machine à café pour avoir un bon petit café serré; c'est un système embarqué.
 - ❑ Vous allumez la télévision et utilisez votre télécommande c'est un système embarqué.

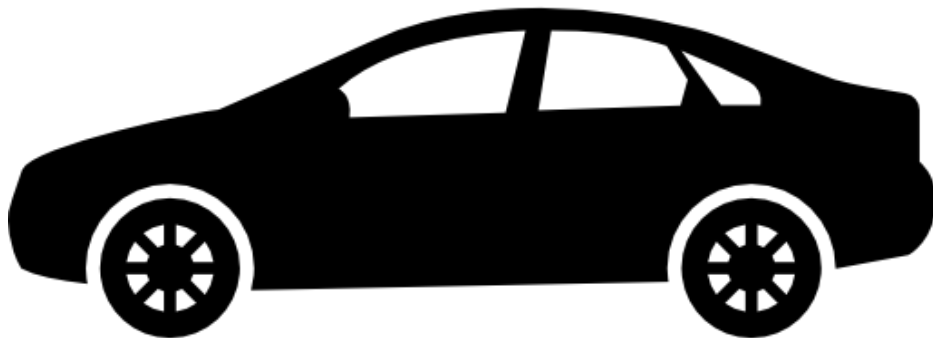


MDA Electromenager
Radio réveil - MDA



Système embarqué

- ❑ En observant notre environnement quotidien, on peut facilement constater la présence de systèmes embarqués.
 - ❑ Vous prenez votre voiture et la voix du calculateur vous dit que vous n'avez pas mis votre ceinture; c'est un système embarqué.
 - ❑ Vous appelez votre ami avec votre téléphone portable pour signaler que vous serez en retard; c'est un système embarqué.
 - ❑ Les systèmes embarqués nous entourent et nous sommes littéralement envahis par eux.



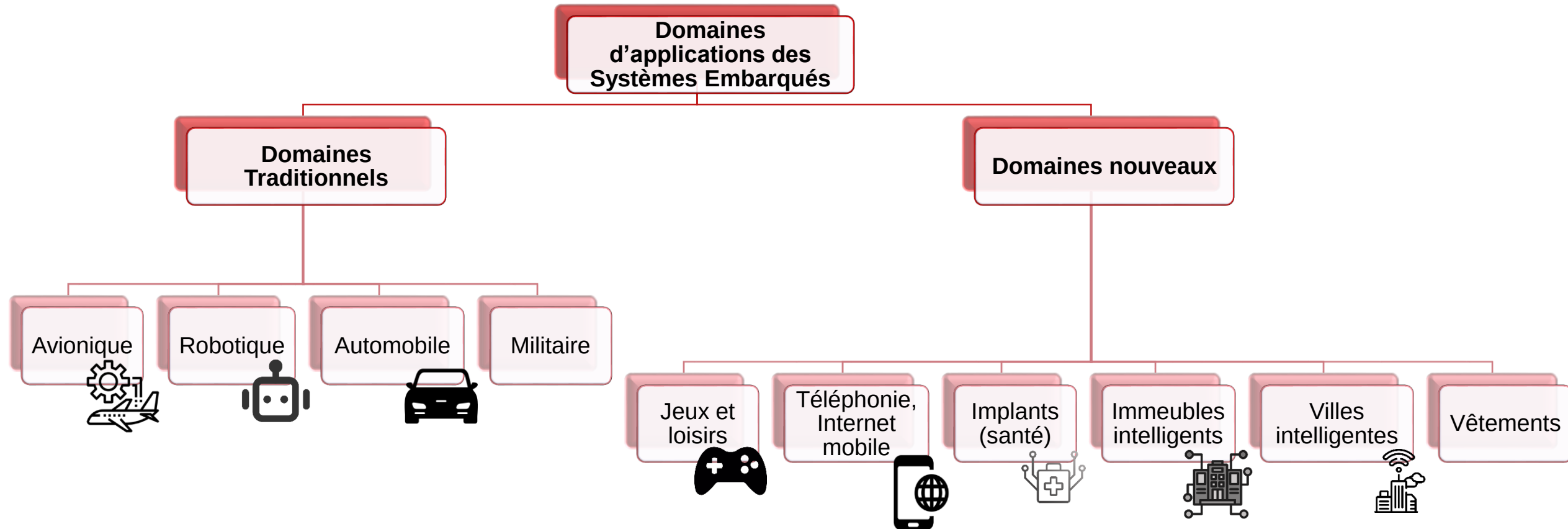
Système embarqué

- ❑ Les systèmes embarqués sont omniprésents dans notre quotidien, intégrés dans des domaines variés comme l'aéronautique, l'automobile, la médecine, les jouets pour enfants et les appareils électroniques grand public (téléviseurs, GPS, téléphones, consoles de jeux). Ils reposent sur des puces électroniques combinant CPU, mémoire et programmes pour des fonctions spécifiques

Quelques exemples :



Domaines d'Application des Systèmes Embarqués



Définition

- i** Un système embarqué est défini comme un système **électronique** et **informatique autonome**, souvent **temps réel**, spécialisé dans une **tâche bien précise**. Ses **ressources** sont généralement **limitées**.
- i** Un système embarqué est un système **complexe** qui intègre du **logiciel** et du **matériel** conçus ensemble afin de fournir des fonctionnalités données pour exécuter une tâche spécifique (contrôle, communication, etc.). Autonome, avec le logiciel dédié.
- i** Un SE constitue une partie intégrante d'un système plus large ou une machine: intégré dans un appareil (voiture, avion, etc.).
- i** Un SE est sans entrées/sorties classiques comme un clavier ou un écran d'ordinateur.
- i** Doté d'un affichage limité (écran LCD) ou parfois sans affichage: Contrairement à un PC, l'interface IHM d'un système embarqué peut être aussi simple qu'un LED qui clignote ou aussi complexe qu'un afficheur à cristaux liquides LCD

1. Système embarqué
2. Caractéristiques d'un système embarqué

Caractéristiques d'un système embarqué

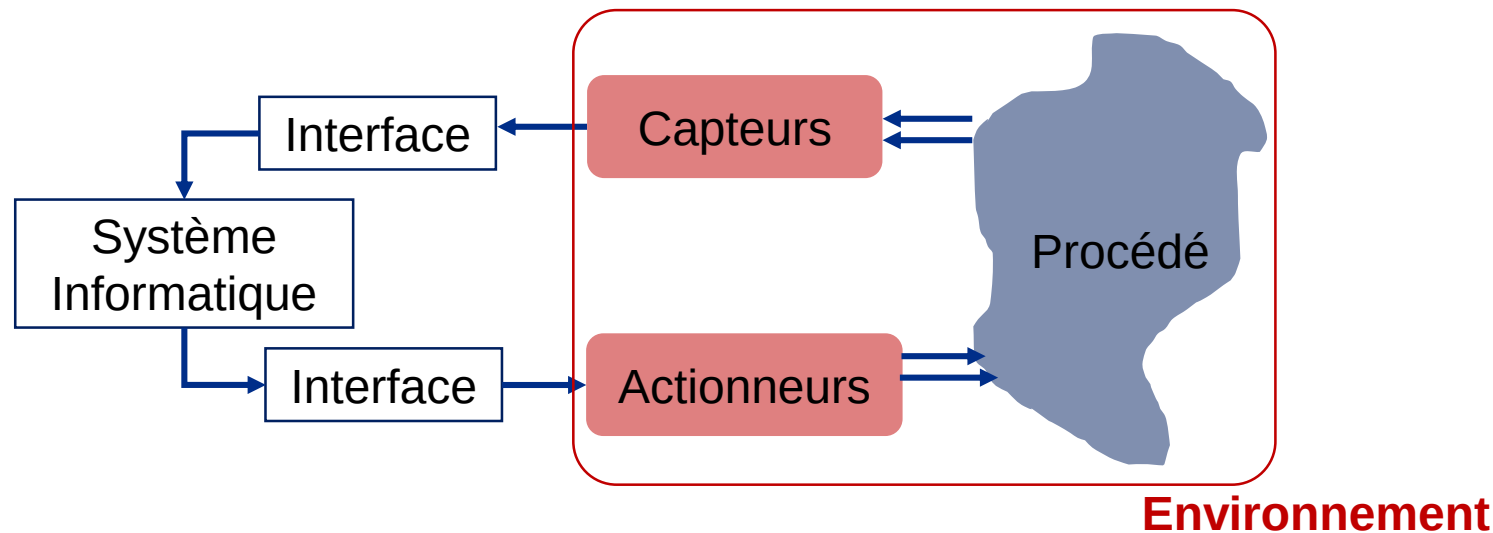
- ❑ Les SEs programmables sont dotés d'interfaces de programmation et leur programmation est une activité spécialisée
- ❑ Le logiciel a une fonctionnalité fixe à exécuter qui est spécifique à une application: L'utilisateur n'a pas la possibilité de modifier les programmes.



Carte de développement STM32F..

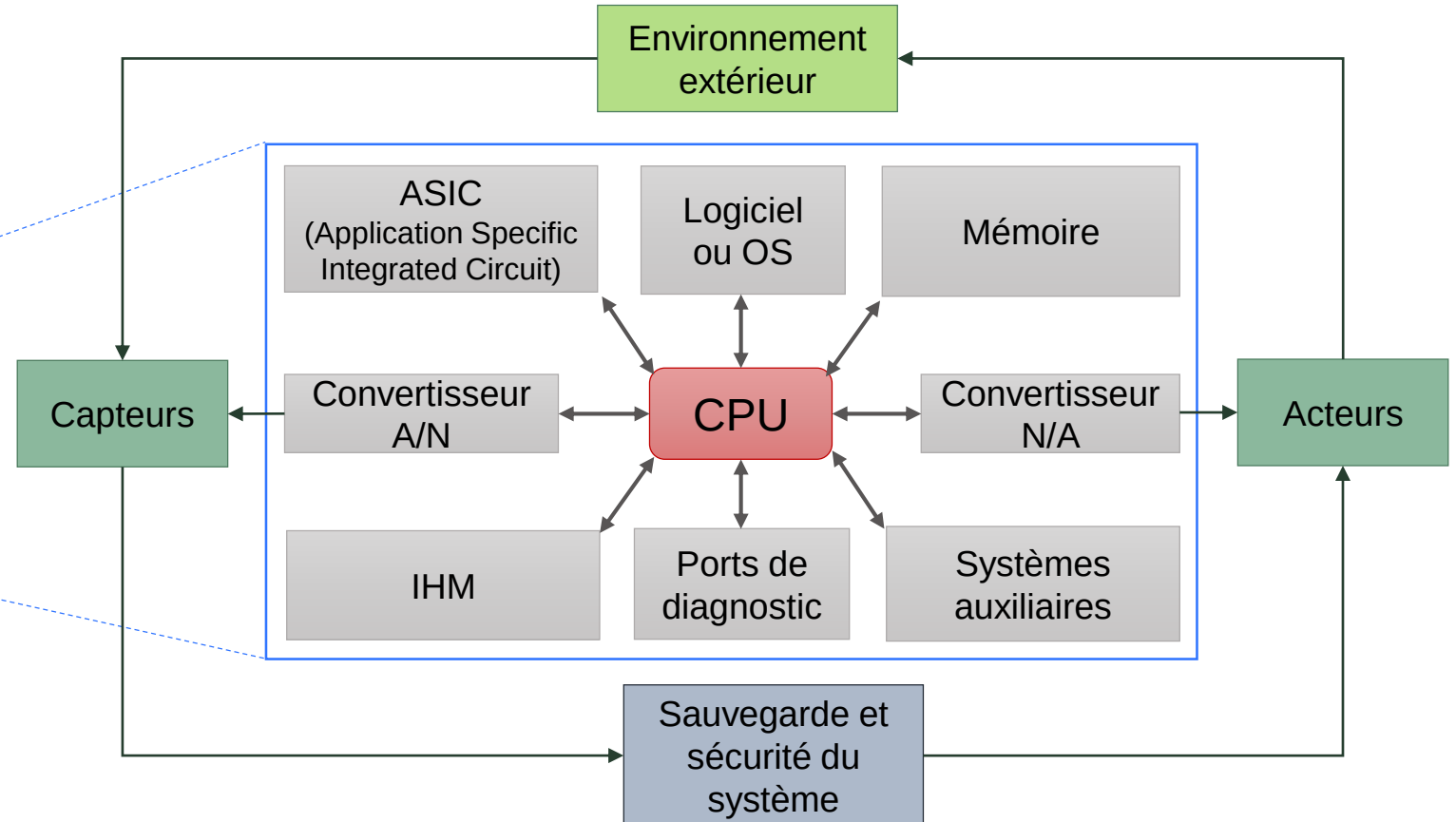
Organisation des SEs

- ❑ **Le système est contrôlé** = environnement (procédé) équipé d'une instrumentation qui réalise l'interface avec le système de contrôle
- ❑ **Le système de contrôle** = éléments matériels (microprocesseurs) et logiciels dont la mission est d'agir sur le procédé via les actionneurs en fonction de l'état de ce procédé indiqué par les capteurs de manière à maintenir ou conduire le procédé dans un état donné

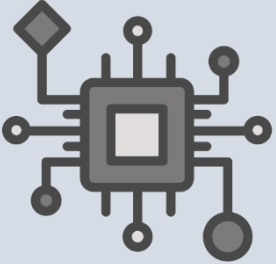


Organisation des systèmes embarqués

La manière de vivre d'un Système embarqué



Comparaison aux systèmes informatiques standards

	Informatique	Embarqué
	❑ Processeur standard ▪ Architecture standard (VanNewmann) ▪ Vitesse élevée (>GHz) ▪ Forte consommation électrique	❑ Processeur standard ▪ Architecture standard adaptée ▪ Vitesse réduite (< 200 MHz) ▪ Basse consommation électrique
	Grande capacité mémoire	Faible capacité mémoire
	OS standard	OS adapté (Embarqué, RTOS, ...)
	Grand nombre de périphériques	Nombre réduit de périphériques

Classification des systèmes embarqués

- ❑ Classification selon mode d'interaction avec l'environnement et la gestion du temps

Système Transformationnel

- **Fonctionnement** : Il lit des données à son démarrage, effectue un calcul et produit un résultat, puis s'arrête.
- **Exemple** : Un compilateur, un convertisseur de fichiers, un système de traitement de signal en boucle fermée.

Système Interactif

- **Fonctionnement** : En interaction quasi permanente avec l'environnement ou un utilisateur. La réaction dépend des événements et de l'état actuel. Le rythme de l'interaction est généralement déterminé par le système lui-même.
- **Exemple** : Un distributeur automatique, un système de gestion de parking, une console de jeux.

Système Réactif (ou Temps Réel)

- **Fonctionnement** : En interaction constante avec l'environnement, ce type de système doit réagir à des stimuli externes dans des contraintes temporelles strictes (temps réel dur ou souple). Le rythme est imposé par l'environnement extérieur.
- **Exemple** : Systèmes de freinage ABS, airbag, pilote automatique d'avion.

Classification des systèmes embarqués

- ❑ Classification selon les exigences de performance/fonctionnalité

Stand-alone (Autonomes)

- Fonctionnent seuls sans hôte (ex: montres numériques, micro-ondes).

Temps réel (Real-time)

- Doivent impérativement respecter des contraintes temporelles strictes (ex: Airbag, systèmes de freinage ABS).

En réseau (Networked)

- Connectés à un réseau (LAN, internet) pour communiquer (ex: domotique, caméras IP).

Mobiles

- Systèmes portables compacts (ex: téléphones, appareils photo, wearables).

Classification des systèmes embarqués

❑ Classification selon le comportement temporel (Déterminisme)

Temps réel strict (Hard Real-Time)

- Une seule échéance manquée rend le système défaillant.

Temps réel souple (Soft Real-Time)

- Des retards minimes sont acceptables (ex: systèmes multimédia).

Sans contrainte temporelle

- Systèmes à temps partagé où la vitesse n'est pas critique.

Classification des systèmes embarqués

❑ Classification selon la complexité et le matériel

Petite échelle

- **Microcontrôleur 8 ou 16 bits, peu de mémoire, tâches simples.**

Moyenne échelle

- **Microcontrôleur 16 ou 32 bits, nécessitant des logiciels plus complexes.**

Grande échelle (Systèmes complexes)

- **Processeurs 32/64 bits, utilisent souvent un OS (RTOS ou Linux embarqué), forte puissance de calcul.**

Classification des systèmes embarqués

❑ Classification selon le mécanisme de déclenchement

Déclenchement événementiel (Event-driven)

- La réponse est déclenchée par un événement externe (interruption).

Déclenchement temporel (Time-driven)

- La réponse est déclenchée selon une planification temporelle prédéfinie.

Classification des systèmes embarqués

❑ Classification selon le domaine d'application

Industriel

- Automates programmables, contrôle-commande.

Grand public

- Électroménager, domotique.

Transport

- Automobile (calculateurs), aéronautique.

Médical

- Pacemakers, imagerie.

Exemples de systèmes d'exploitation embarqués

Propriétaires	Open Sources
☐ VxWorks ☐ Windows CE, XP embedded, ☐ Symbian ☐ Nucleus ☐ ENEA OSE Delta ☐ Qualcomm OS ☐ ThreadX ☐ VRTX ☐ EROS ☐ GreenHills IntegrityOS ☐ mITRON NORTi	☐ eCos FreeRTOS ☐ Embedded Debian ☐ AMSEL ☐ Embedded Gentoo ☐ ETLinux ☐ FREESCO ☐ Linux Router Project ☐ Linux-VR Project ☐ Linux On A Floppy ☐ Qplus ☐ Midori Linux ☐ uClinux ☐ µLinux ☐ PeeWeeLinux

Plan du cours

Définition et généralités

Conception des systèmes embarqués

Conception matérielle

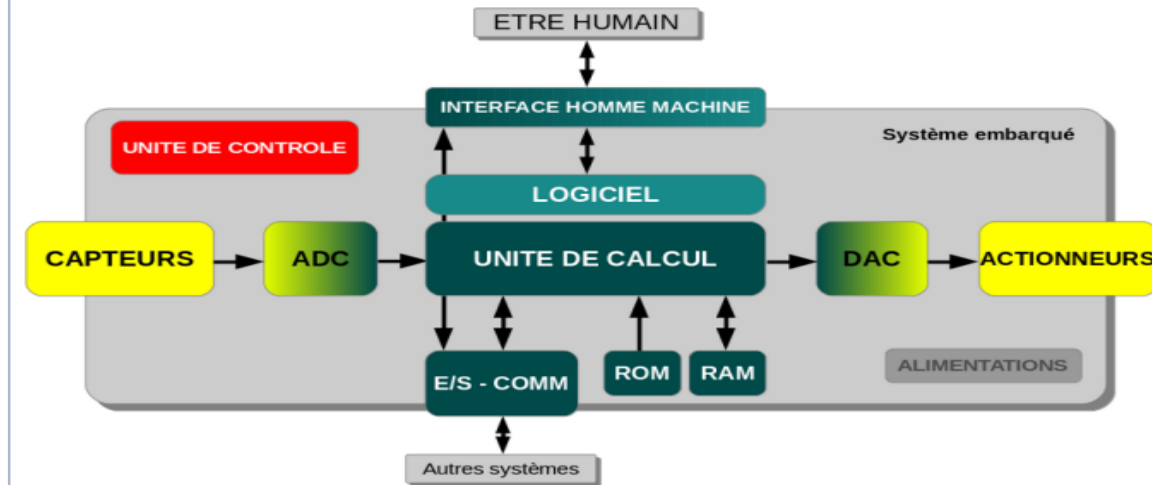
Conception logicielle

Conception Mixte

Résumé

Conception des systèmes embarqués

1. Unité de Calcul (CPU) est le cerveau du système embarqué, il exécute toutes les opérations du système
2. Mémoire: La mémoire est utilisée pour stocker les données, les programmes, les paramètres et d'autres informations essentielles pour le fonctionnement du système, Elle se divise souvent en deux types: Mémoire vive (RAM): utilisée pour le stockage temporaire des données en cours d'utilisation, Mémoire morte (ROM ou Flash): Stocke les programmes et les données permanentes.
3. Capteur: Il mesure la qualité physique et la convertit en un signal électrique qui peut être lu par un observateur ou par n'importe quel instrument électronique comme un convertisseur A/N. Le capteur stocke la quantité mesurée dans la mémoire.
4. Actionneur: Un actionneur compare la sortie fournie par le convertisseur (N/A) à la sortie réelle (attendre) qui y est stocké et stocke la sortie approuvée.
5. Convertisseur Analogique/Numérique (ADC): Un convertisseur (A/N) convertit le signal analogique envoyé par le capteur en un signal numérique.
6. Convertisseur Numérique/Analogique (N/A): Un convertisseur numérique-analogique convertit les données numériques fournies par le processeur en données analogiques



Principales caractéristiques des systèmes embarqués

- ❑ Processeurs basse consommation avec fonctionnalités minimum (**faible puissance de calcul**)
- ❑ Mémoire vive et mémoire de masse de **taille limitée**
- ❑ Poids et volume **réduit**
- ❑ Mobilité
- ❑ Communication limitée: **La communication affecte la batterie**
- ❑ Réactifs: Il doit **réagir** à l'arrivée d'informations extérieures non prévues
- ❑ C'est un système principalement **numérique**

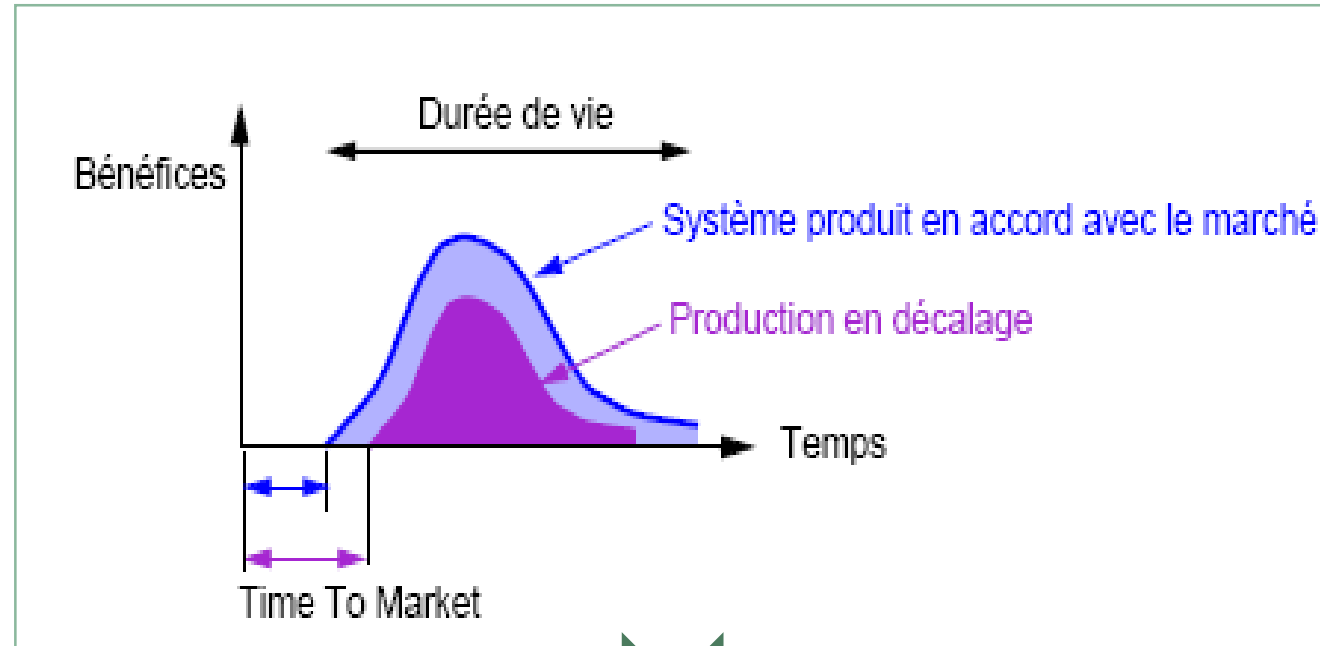
Contraintes de conception des systèmes embarqués

- ❑ **Performance:** puissance de calcul MIPS (Million d'Instructions Par Seconde) → temps d'exécution
- ❑ **Fiabilité:** la fiabilité est l'aptitude d'un système à accomplir une fonction requise dans des conditions données pour une période de temps donnée (le système est utilisé sans défaillance pendant cette période).
- ❑ **Surface et encombrement:** GSM...
- ❑ **Consommation énergétique:** lié à l'autonomie des batteries (PDA: personal digital assistant, téléphone mobile...)
- ❑ **Sûreté:** Capacité à accorder une confiance justifiée au service
- ❑ **Robustesse:** aucun dommage (automobile, avionique...) grâce à la résistance aux chocs, à la température, à l'humidité
- ❑ **Coût et temps de développement:** faible coût
- ❑ **Résistance aux pannes:** La panne d'un composant ne remet pas en cause la vie du système
- ❑ **Encombrement mémoire réduit:** Mémoire limitée, pas de disque en général

Contraintes de conception des systèmes embarqués

- ❑ Un système embarqué doit **répondre à des contraintes temps réel**:
 - ❑ Un système temps réel doit réagir à un stimuli (événement externe) dans un intervalle de temps dépendant de l'environnement de travail.
 - ❑ Un système temps réel qui produit une bonne réponse mais trop tard est défaillant.
 - ❑ Contrainte de temps réel: **dure** ou **souple**.
 - ❑ Une contrainte temps réel est appelé dure si le non respect de cette contrainte peut mener à des situations critiques voir catastrophiques
 - ❑ Les autres contraintes sont appelées soft

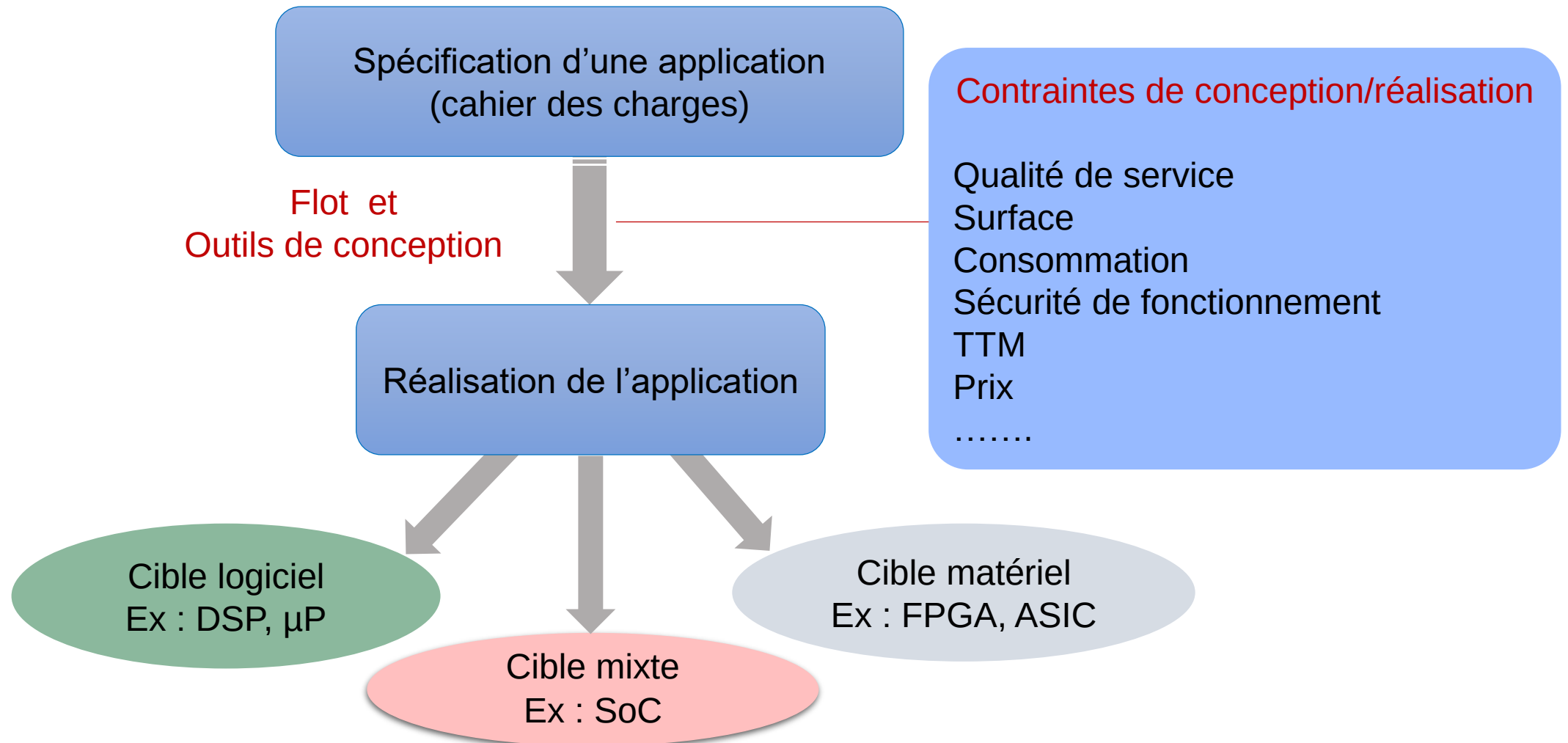
Contraintes de conception des systèmes embarqués



Les produits ont une durée de plus en plus faible:

- Réduire le « time to market »
- Réutilisation pour concevoir d'autres produits (rentabiliser)

Conception des systèmes embarqués et ses contraintes



Plan du cours

Définition et généralités

Conception des systèmes embarqués

Conception matérielle

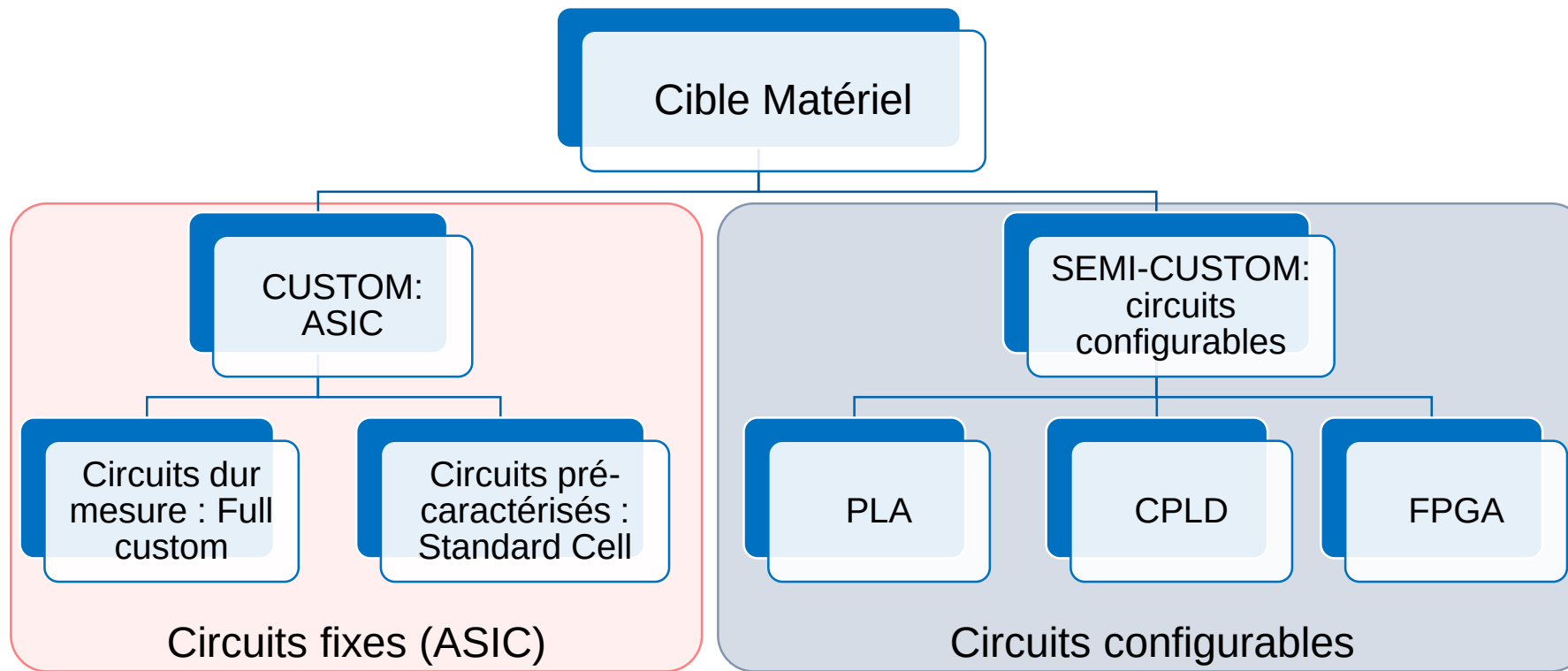
Conception logicielle

Conception Mixte

Résumé

1. Cible Matériel
2. Technologie de base: Les transistors
3. Evolution des systèmes embarqués
4. Exemples de Systèmes Embarqués

Cible Matériel

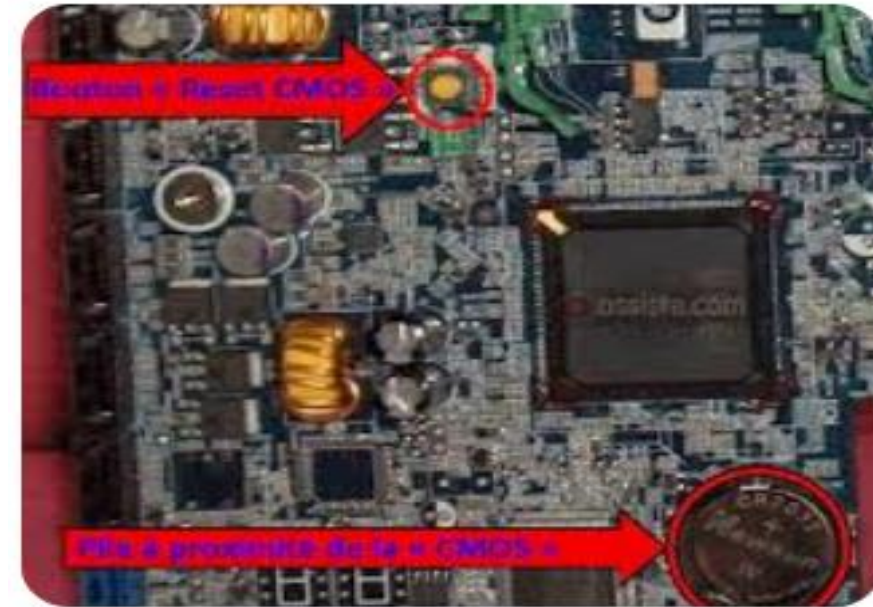
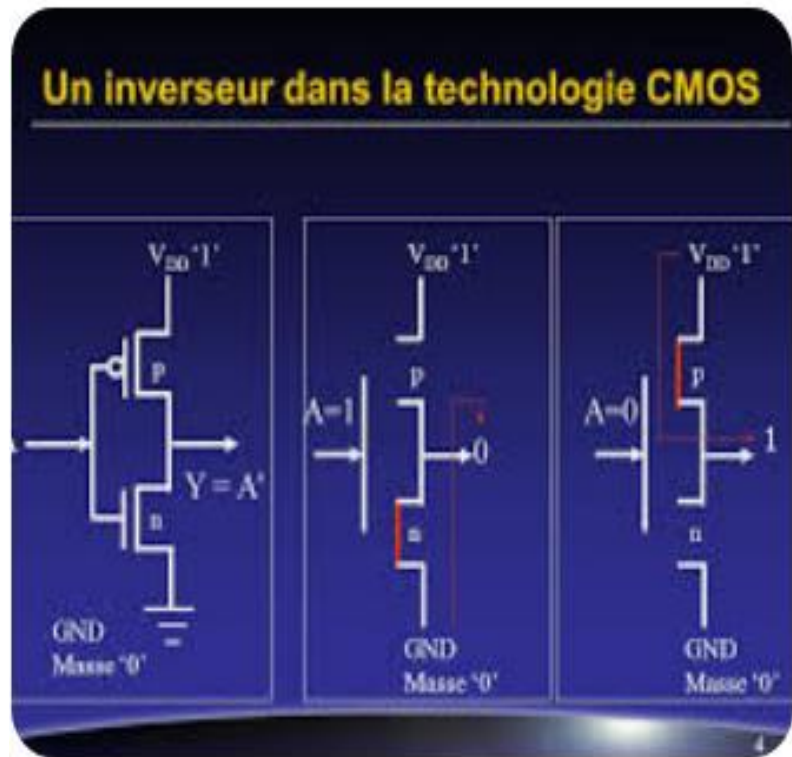


❑ **ASIC:** Application Specific Integrated Circuit

❑ **FPGA:** Field Programmable Gate Array

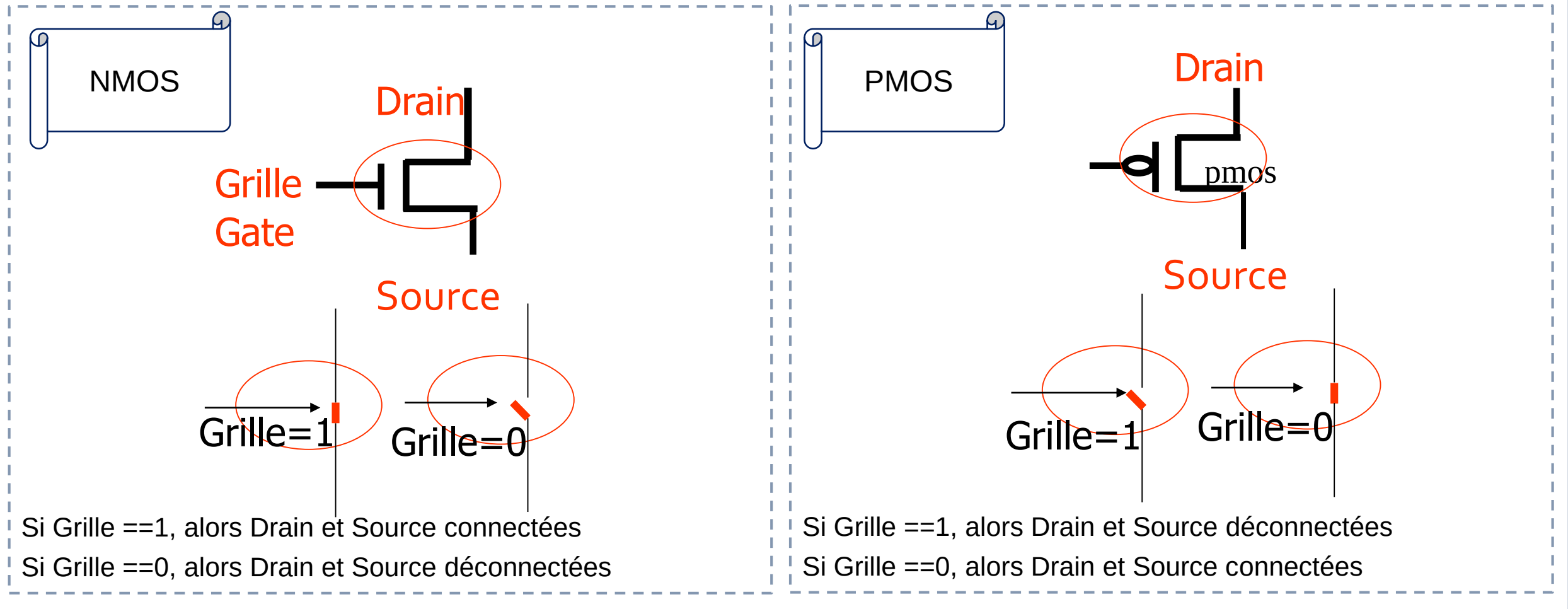
1. Cible Matériel
2. Technologie de base: Les transistors
3. Evolution des systèmes embarqués
4. Exemples de Systèmes Embarqués

Cible Matériel



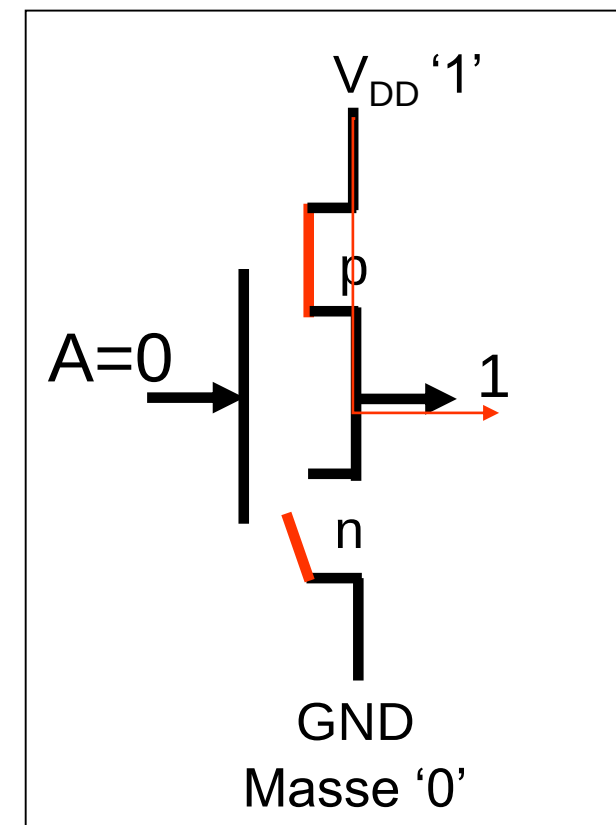
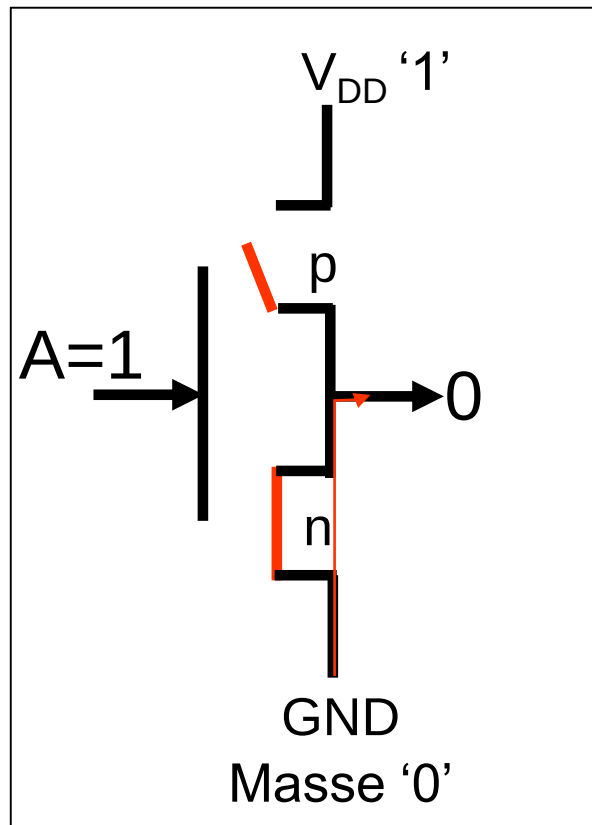
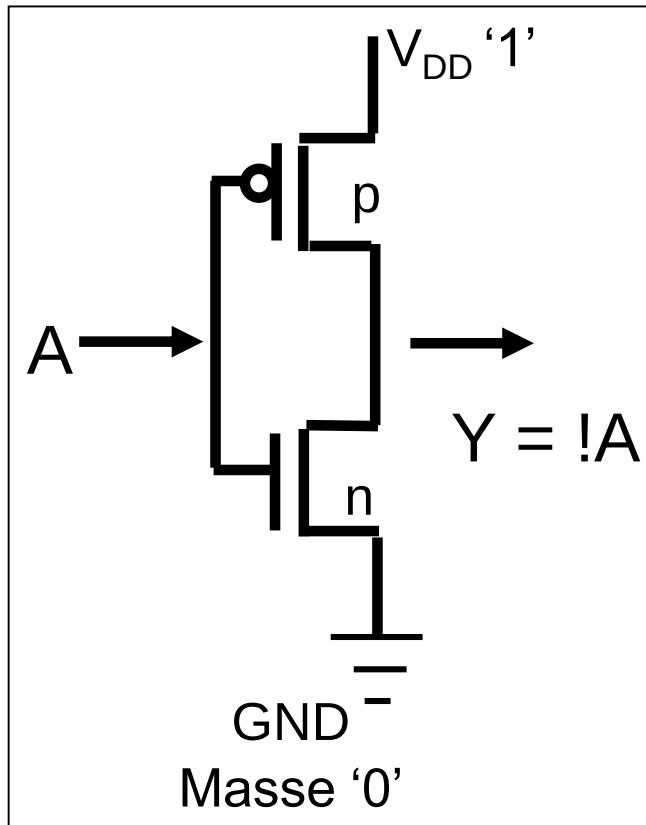
CMOS

Les Transistors



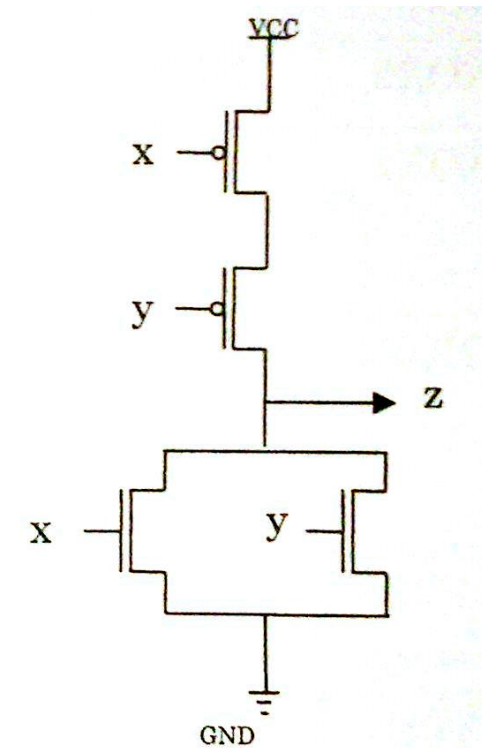
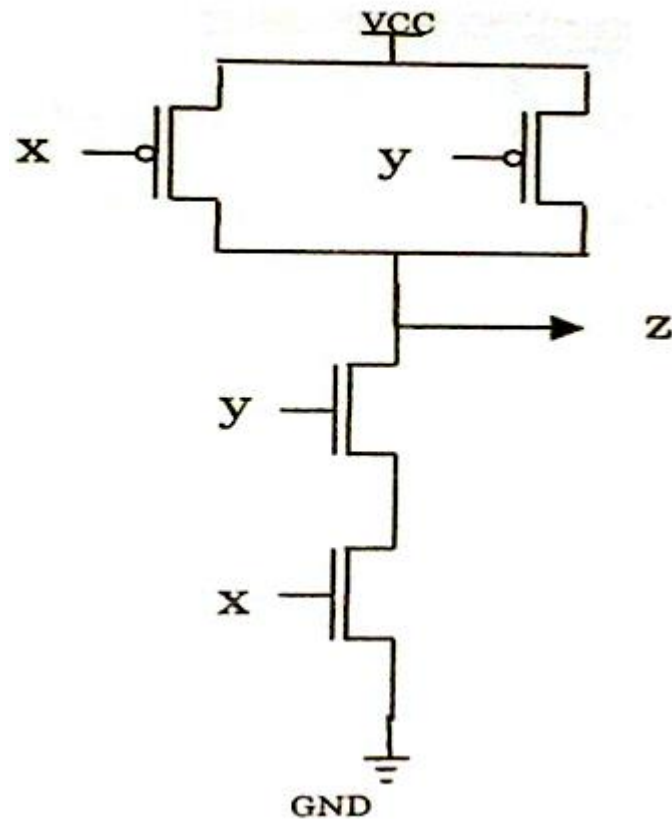
Les Transistors

CMOS : Complementary MOS (circuit comportant des PMOS et des NMOS)

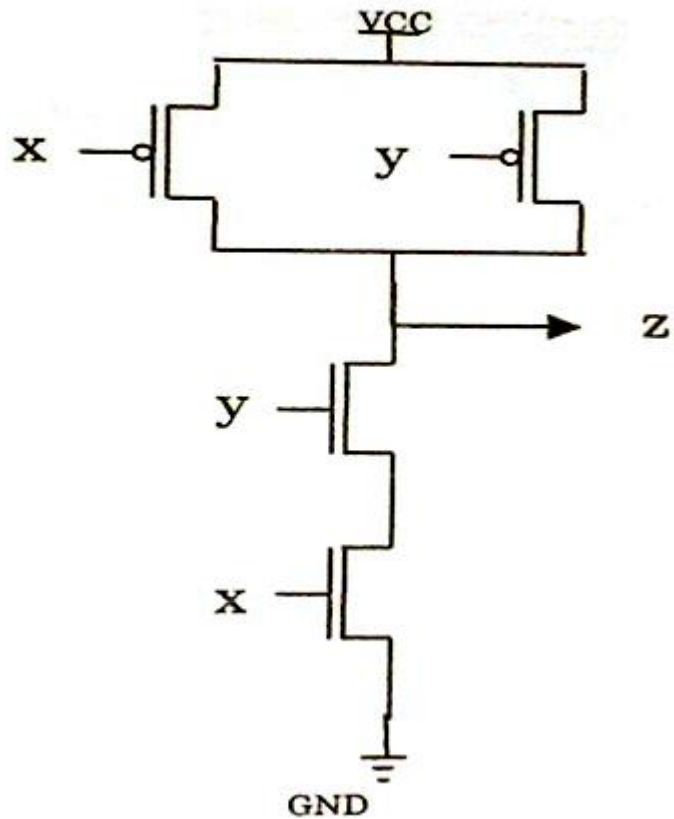


Exercice 1

De quel type de porte s'agit-il ?



Exercice 1: Solution

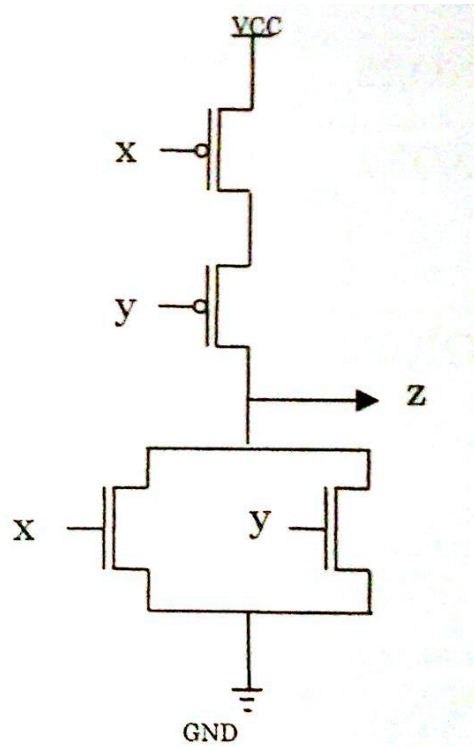


$$Z = \text{NAND}(x,y)$$

x	y	z
0	0	1
0	1	1
1	0	1
1	1	0

→ Une porte NAND en technologie CMOS

Exercice 1: Solution



$$z = \text{NOR}(x, y)$$

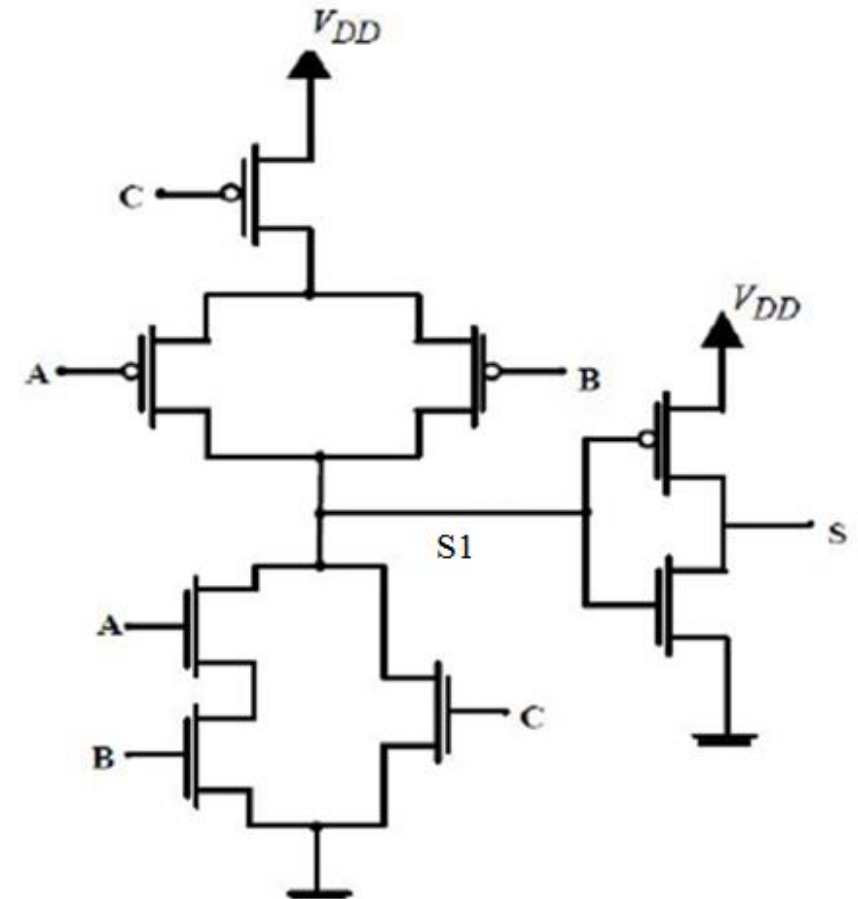
x	y	z
0	0	1
0	1	0
1	0	0
1	1	0

→ Une porte NOR en technologie CMOS

Exercice 2

Donner la fonction logique de ce circuit CMOS

$$S = f(A, B, C) = AB$$

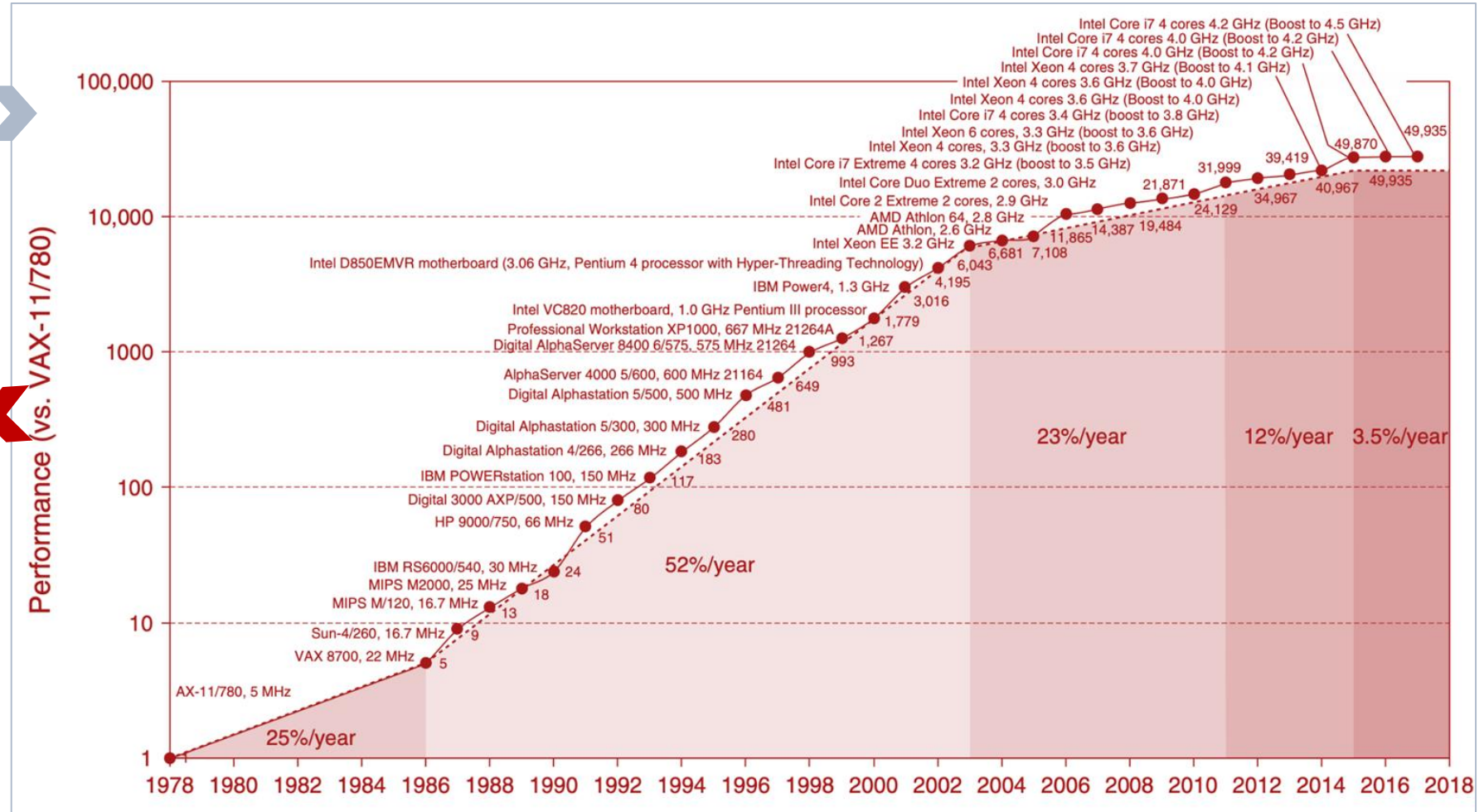


1. Cible Matériel
2. Technologie de base: Les transistors
3. **Evolution des systèmes embarqués**
4. Exemples de Systèmes Embarqués

Loi de Moore

❑ « le nombre de transistors dans un semiconducteur double tous les 2 ans »
Moore

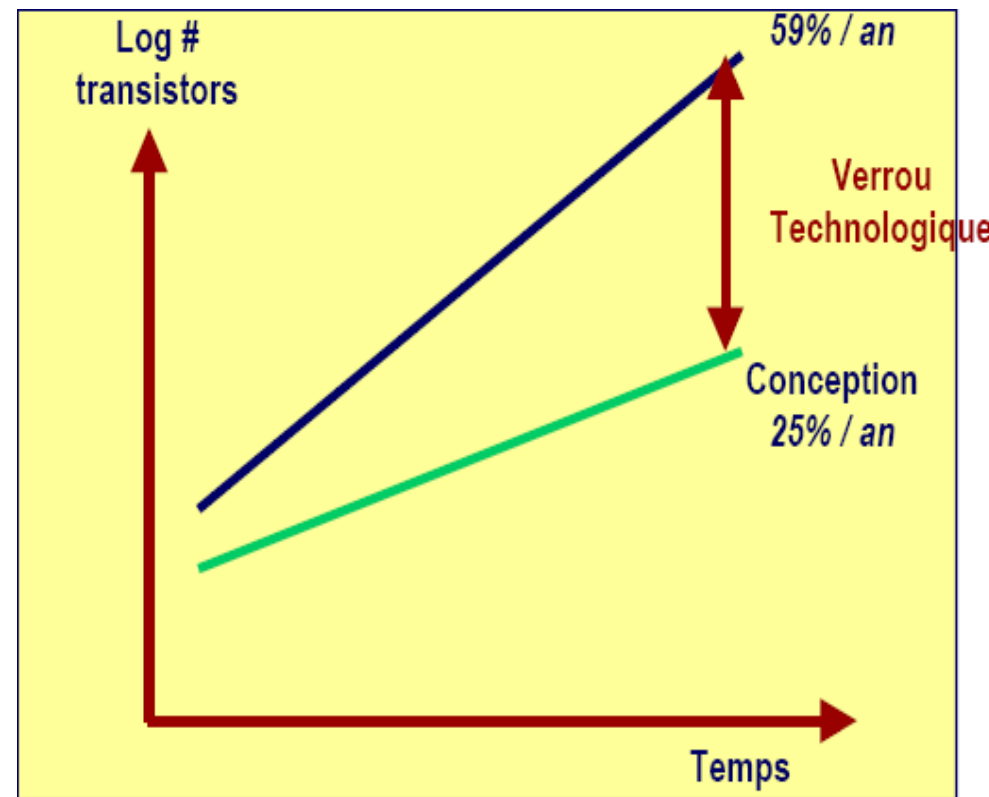
❑ Depuis 2015, les processeurs de type x86 (qui équipent les PC et les serveurs et descendent de l'Intel 4004) ne progressent quasiment plus en performance.



[Computer Architecture: A Quantitative Approach, sixth edition - p. 3, figure 1.1]

Evolution technologique/conception

Le processus
technologique autorise un
accroissement de la
complexité de 59% par an
→ Loi de MOORE

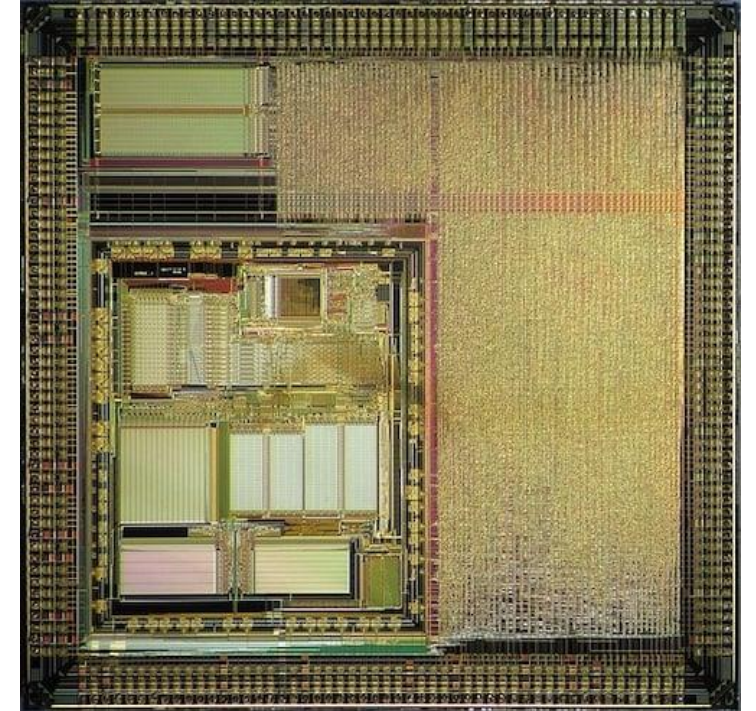


L'efficacité des
concepteurs n'augmente
que de 25% par an

Les ASICs: Application Specific Integrated Circuits

❑ Definition:

- ❑ ASIC signifie Application Specific Integrated Circuit (circuit intégré spécifique à une application).
- ❑ Il s'agit d'une puce électronique spécialisée qui regroupe sur un même composant tous les éléments nécessaires pour exécuter une fonction précise.
- ❑ Contrairement aux circuits standards, un ASIC est exclusivement dédié à une application ou un utilisateur



Exemple d'ASIC comportant un DSP (Digital Signal Processor) [Wiki Commons and Pauli Rautakorpi]

Les ASICs: Application Specific Integrated Circuits

Caractéristiques principales

Objectif : réaliser une fonction spécifique (ex. traitement réseau, cryptographie, image, audio).

Architecture : intègre directement les composants actifs nécessaires sur une seule puce.

Exemples d'utilisation

Télécommunications : gestion des signaux dans les réseaux.

Cryptomonnaies : calcul intensif pour le minage de Bitcoin.

Neurosciences et traitement du signal : séparation de sources sonores ou d'images.

Électronique grand public : processeurs graphiques ou puces de décodage vidéo

Les ASICs: Application Specific Integrated Circuits



Avantages

- Hautes intégrations
- Hautes performances (vitesse, low-power)
- Coûts faibles pour de gros volumes de production
- Personnalisation: Performance optimisée pour la tâche ciblée.



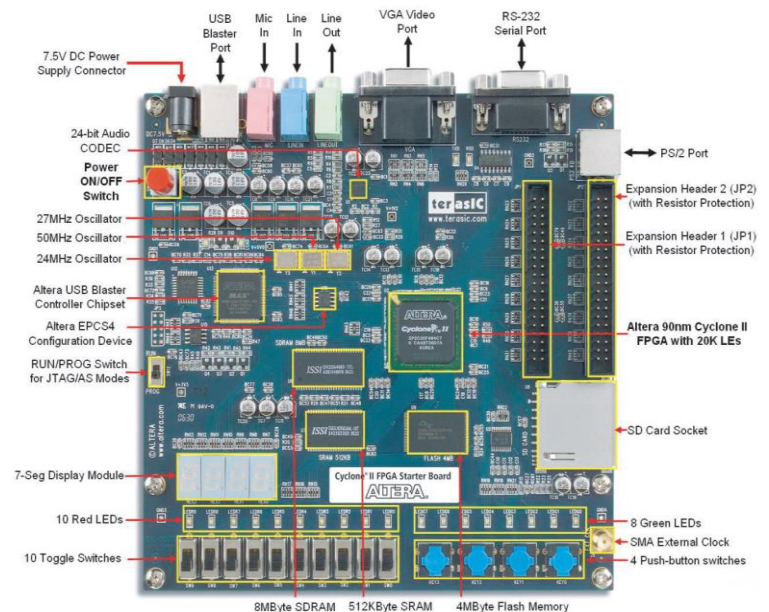
Inconvénients

- Prix du 1er exemplaire
- Non-flexible
- Time-to-market élevé
- Fabrication réservée aux spécialistes (fondeur)

Les FPGAs: Field Programmable Gate Arrays

❑ Definition:

- ❑ **FPGA** signifie *Field-Programmable Gate Array* (réseau de portes programmables sur site).
- ❑ C'est un circuit intégré qui contient des blocs logiques configurables (programmables), ainsi que des interconnexions configurables entre ces blocs.
- ❑ Contrairement à un **ASIC** (Application Specific Integrated Circuit), qui est figé pour une tâche unique, le FPGA reste **flexible et reconfigurable**



Exemple d'un FPGA

La configuration du circuit, c'est-à-dire la programmation de sa fonction, se fait sur place (Field Programmable), sans envoi chez un fabricant

Les FPGAs: Field Programmable Gate Arrays

Caractéristiques principales

Programmabilité : l'utilisateur définit l'architecture interne via un langage de description matériel (HDL, comme VHDL ou Verilog).

Structure : Composé de cellules logiques (portes, bascules, multiplexeurs) reliées par des interconnexions programmables

Exemples d'utilisation

Prototypage matériel : tester une architecture avant de fabriquer un ASIC.

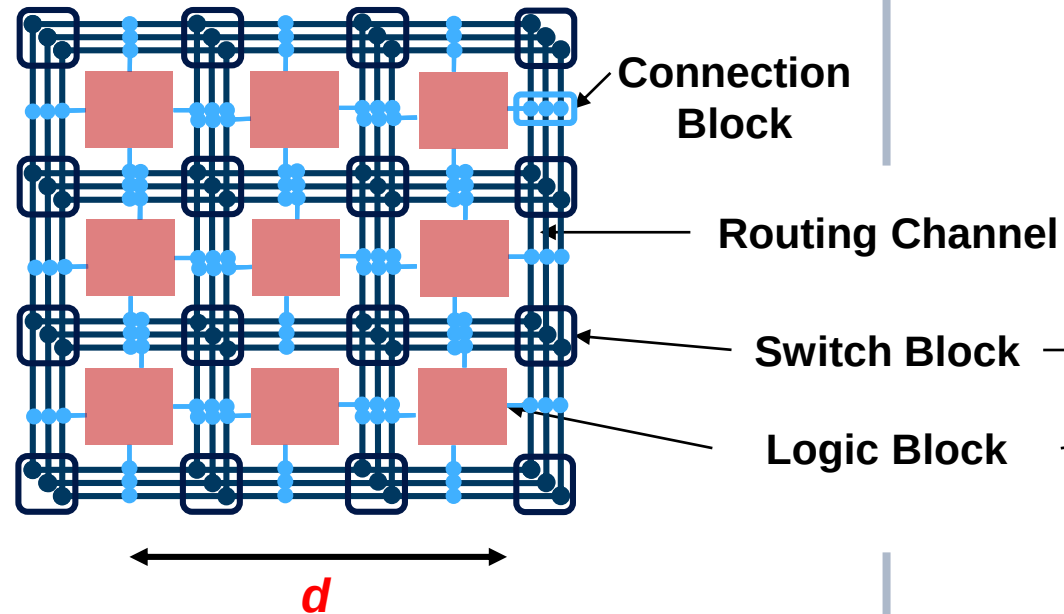
Télécommunications : traitement de signaux numériques (DSP, modulation, codage).

Aéronautique et spatial : systèmes embarqués nécessitant reprogrammation en mission.

Cryptographie et IA : accélération de calculs parallèles.

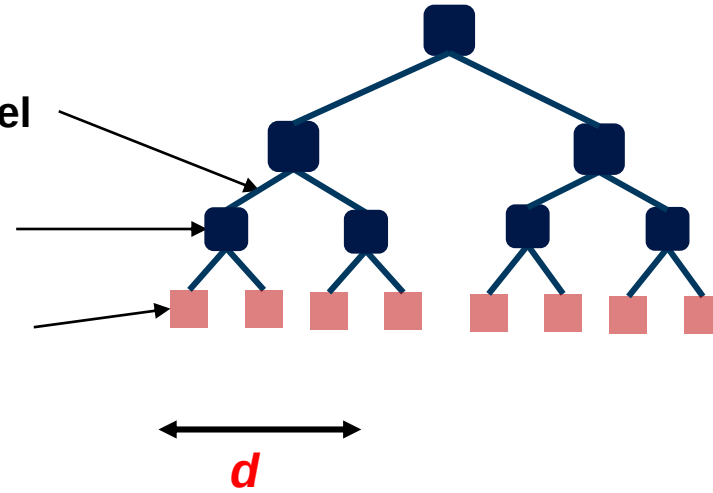
Types d'architectures FPGAs

Architecture Matricielle



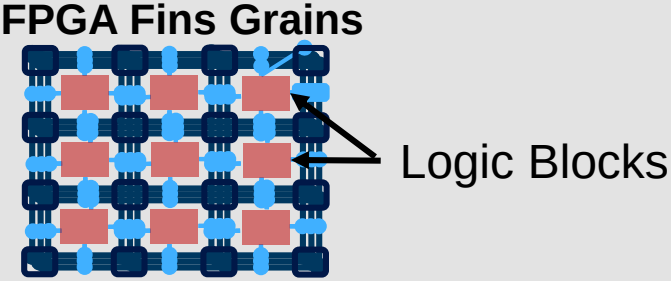
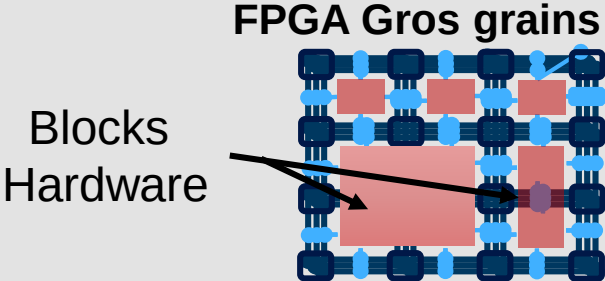
- ❑ Les Blocs Logiques sont arrangé sous forme de matrices 2D et sont entourés par un réseau de routage
- ❑ Le numéro de segments en série augmente linéairement en fonction de d

Architecture Hiérarchique

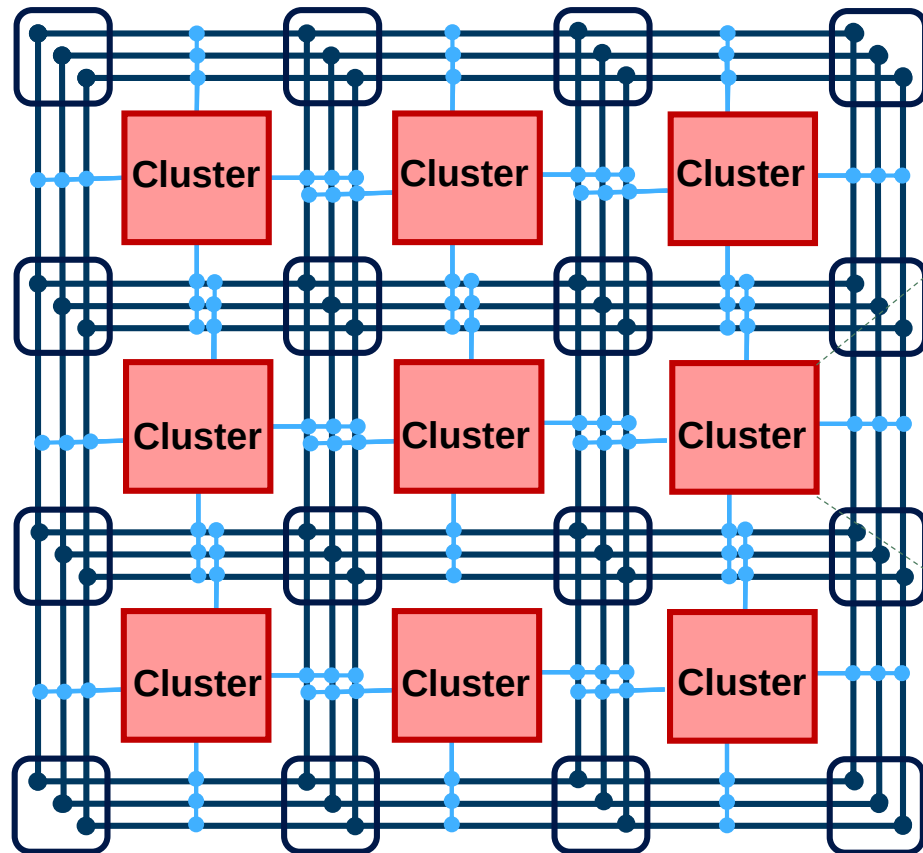


- ❑ Les Blocs Logiques sont connectés récursivement pour former une structure hiérarchique
- ❑ Le numéro de segments en série augmente logarithmiquement en fonction de d

Types d'architectures FPGAs

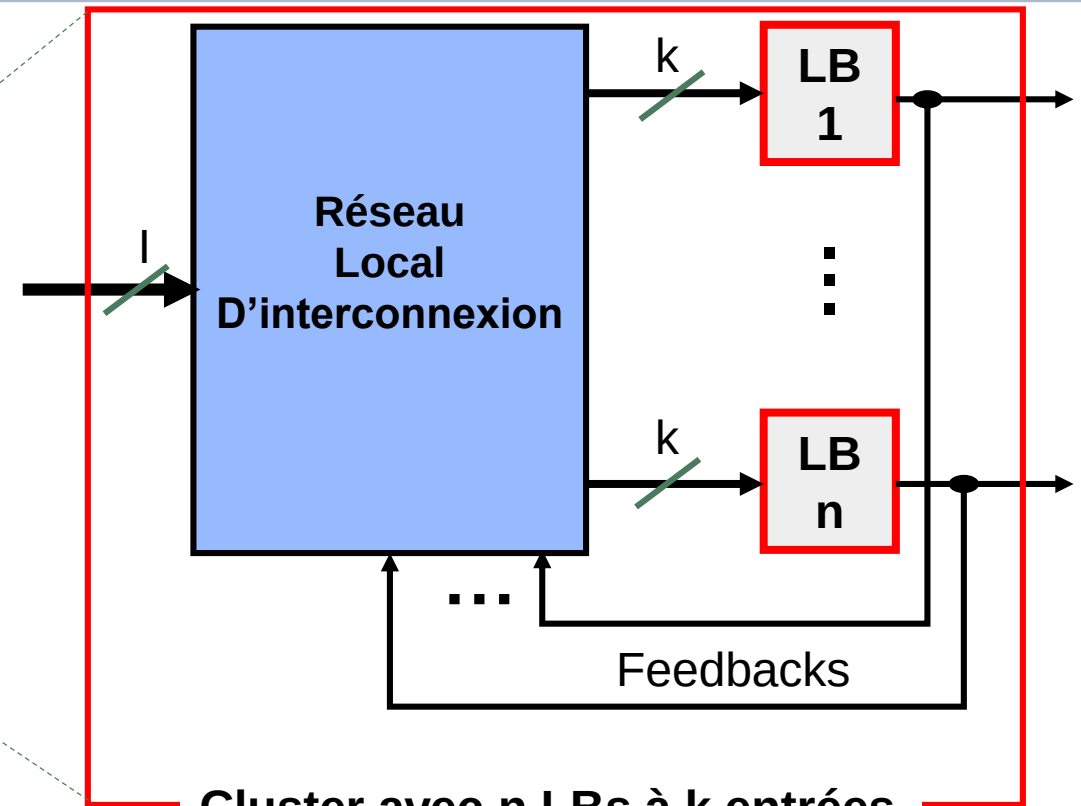
FPGA Granularité	Types d'architectures FPGAs	
	FPGA Fins Grains	FPGA Gros grains
Exemples	 Logic Blocks ✓ Flexibilité importante ✗ Augmentation de puissance, surface et délai	 Blocks Hardware ✓ Meilleure performance ✗ Optimisés pour un domaine d'application particulier
FPGA Matricielle	▪ VPR ▪ Synthesizable Programmable Core: SPC ▪ ArcticPro → 2D	▪ Menta ▪ Xilinx Virtex ▪ Xilinx 7 Series: ▪ Spartan-7 Family ▪ Artix-7 Family ▪ Kintex-7 Family ▪ Virtex 7 → 2D → 3D
FPGA Arborescent	▪ FPGA hiérarchique avec des SBs et canaux de routage optimisés: MFPGA → 2D	▪ EFlex → 2D

Architectures FPGAs: FPGA Matricielle Symétrique



Architecture FPGA Matricielle

- ❑ Exploite le partage de signaux entre LBs localement
- ❑ Améliore les performances globales du FPGA



Cluster avec n LBs à k entrées

- ❑ I : Bande passante d'entrée du cluster
- ❑ k : Bande passante d'entrée du LB
- ❑ n : Taille de Cluster

Architectures FPGAs: Exemple de FPGA Matricielle Symétrique

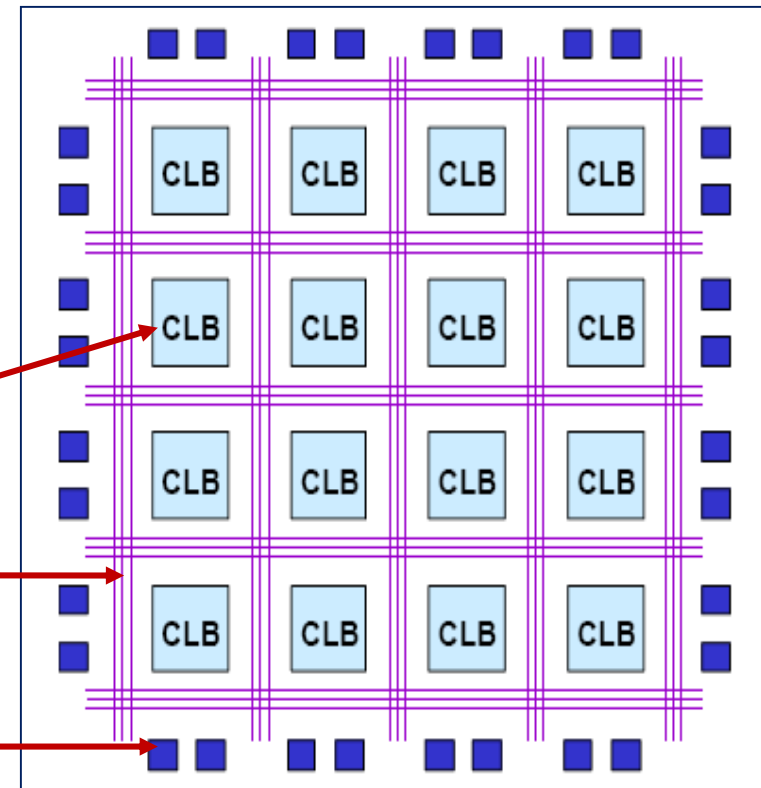
❑ Architecture symétrique

- ❑ Exemple de structure interne d'un FPGA de type matrice symétrique: les FPGA de la série XC4000 de Xilinx.

Un réseau de blocs de logique programmable (Configurable Logic Block: CLB), chaque bloc pouvant réaliser des fonctions combinatoires et séquentiels

Un réseau d'interconnexions programmables entre les blocs
(Programmable switching matrices, PSM)

Des blocs spéciaux d'entrée et de sortie avec le monde extérieur (Input/Output Block –IOB).



Architectures FPGAs: Exemple de FPGA Matricielle Symétrique

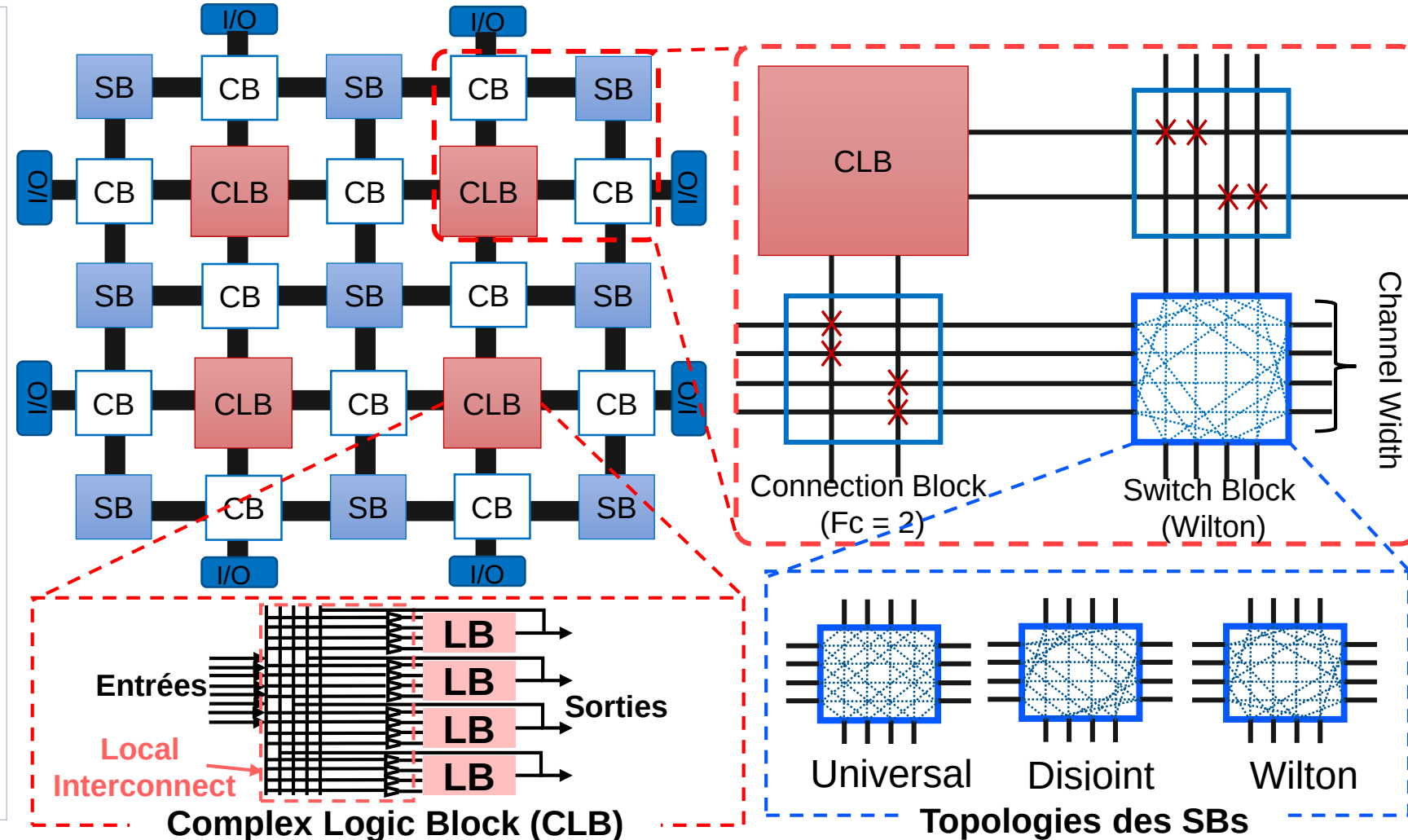
Architecture FPGA VPR

CB dépeuplé:

- ✓ Surface plus rentable.

Crossbar intra-cluster pleinement peuplé:

- ✓ Flexibilité importante.
- ✗ Pas de profit de l'équivalence logique des entrées des LB.
- ✗ Taux d'usage des d'interconnexions des switches est très faible (~12%).
- ✗ Overhead en terme de puissance et surface.



FPGAs: Environnement de développement

❑ FPGA Environnement d'exploration comporte:

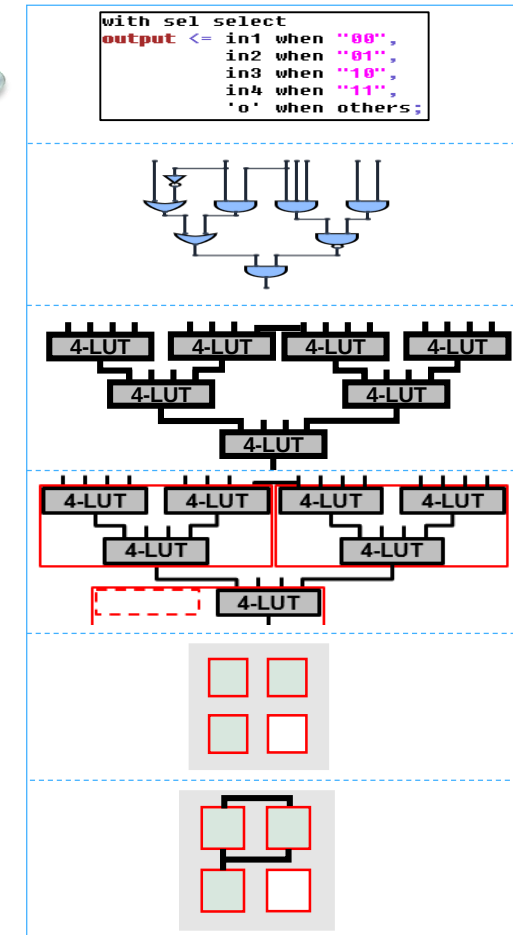
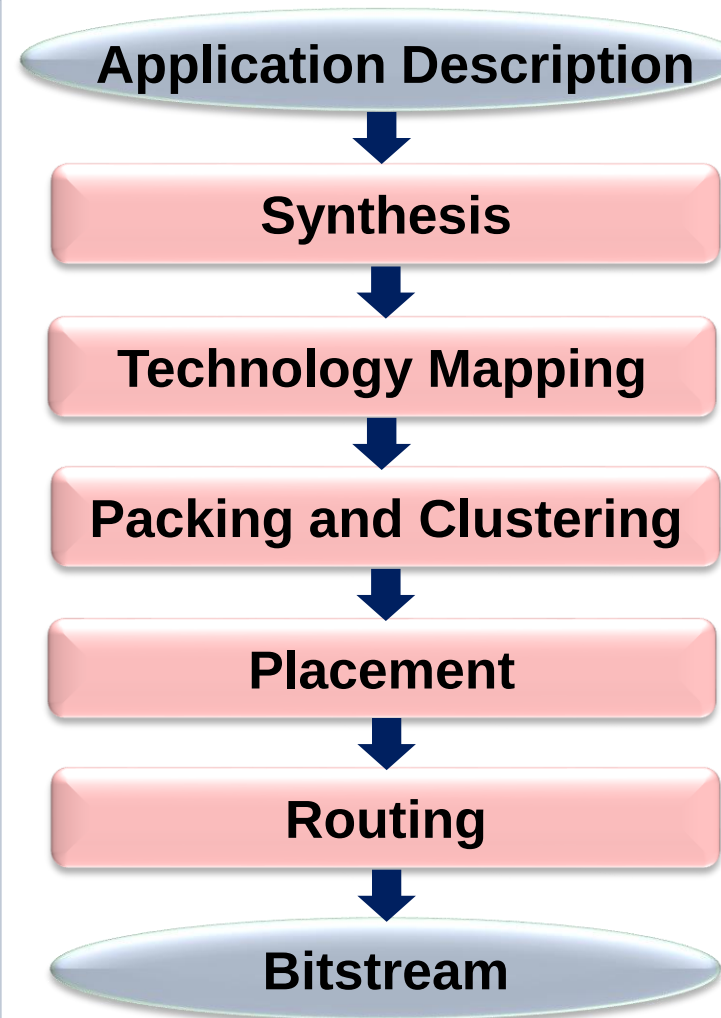
- Outils de configuration pour implémenter automatiquement des applications complexes dans FPGA cible.
- Modèles métriques to pour estimer la performance de FPGA.

❑ Outils de configuration:

- Convertit circuit haut niveau à un bitstream de configuration qui peut être flasher sur FPGA.

❑ Modèles métriques:

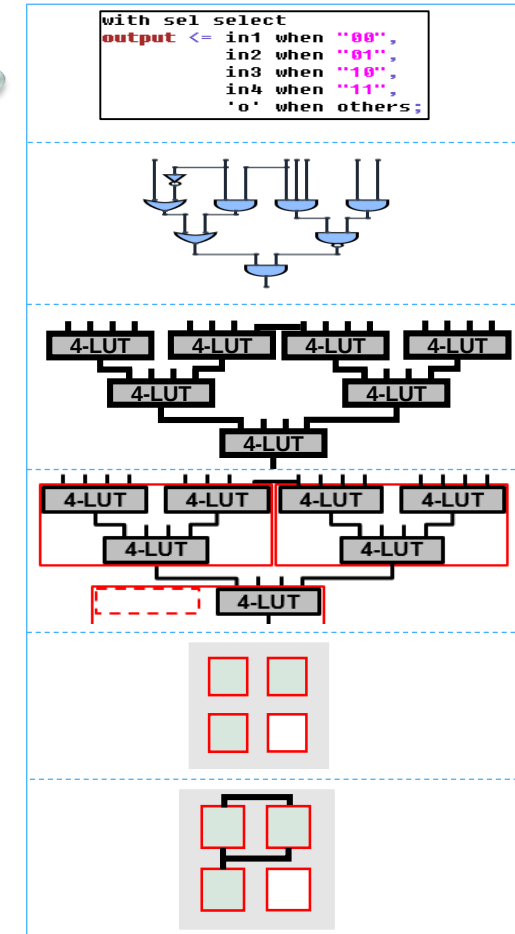
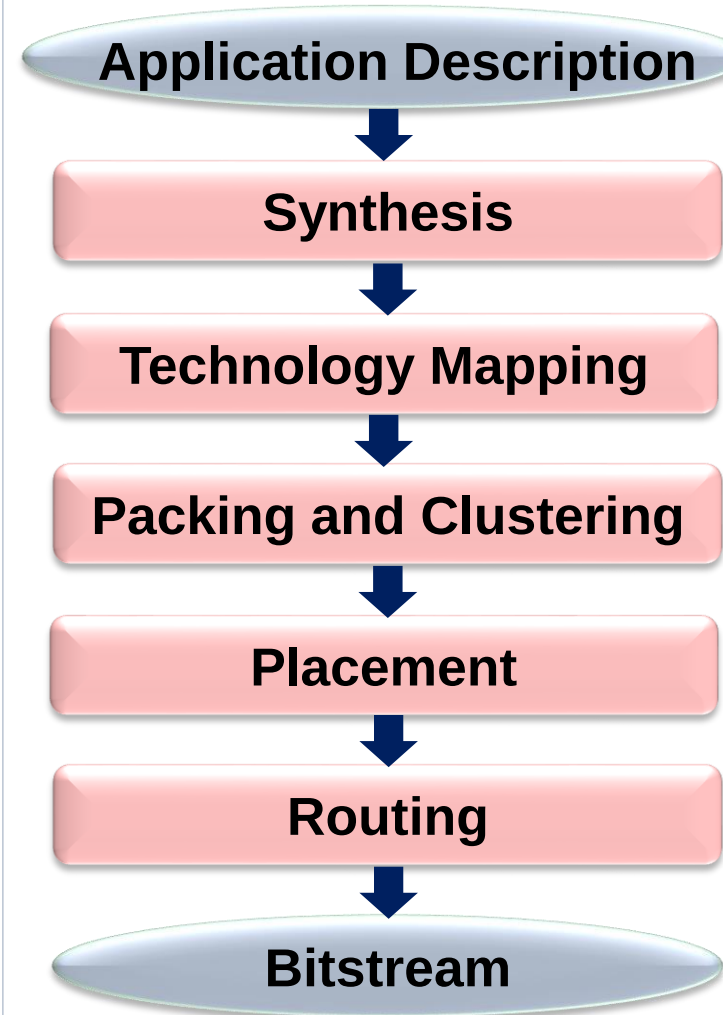
- Consommation de puissance
- Surface
- Performance



FPGAs: Environnement de développement

❑ FPGA Configuration tools Examples:

FPGA	CAD Tools
VPR	▪ VTR CAD Flow
Virtex6	▪ ISE Design Suite
Xilinx 7 Series	▪ Vivado Design Suite
Menta	▪ Origami Programmer Design Flow
SPC	▪ Adicsys CAD Tools
ArcticPro	▪ Aurora CAD Tools
EFlex	▪ Eflex CAD Tools



Les FPGAs: Field Programmable Gate Arrays



Avantages

- Grande flexibilité (reprogrammation possible même après déploiement).
- Idéal pour prototypage et tests rapides.
- Permet d'implémenter des architectures parallèles très performantes.



Inconvénients

- Moins performant et plus coûteux en énergie qu'un ASIC pour une tâche unique.
- Surface et vitesse limitées par rapport aux circuits spécialisés.

VHDL

❑ VHSIC (Very High Speed Integrated Circuit) Hardware Description Language:

❑ Langage qui décrit: La structure et le comportement de systèmes électroniques

❑ Il a été développé en 1981 par le département de défense américaine pour décrire du « hardware »

❑ Il a été standardisé en 1987 par l'IEEE et mis à jour régulièrement

❑ VHDL a été d'abord adopté pour la simulation ensuite la synthèse logique

❑ C'est un langage type comme le C, il faut déclarer le type de chaque signal manipulé

❑ VHDL n'est pas un langage de programmation c'est un langage de description (spécification) de système.

1. Cible Matériel
2. Technologie de base: Les transistors
3. Evolution des systèmes embarqués
4. Exemples de Systèmes Embarqués

VHDL

❑ Exemple : Un INVERSEUR.

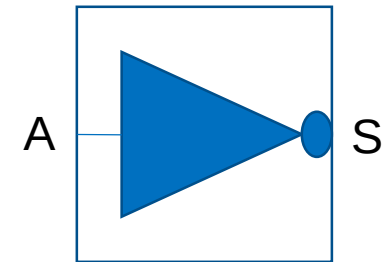
```
library ieee;  
Use ieee.std_logic_1164.all;
```

Déclaration des bibliothèques

Déclaration de l'entité du décodeur
Correspondance schématique

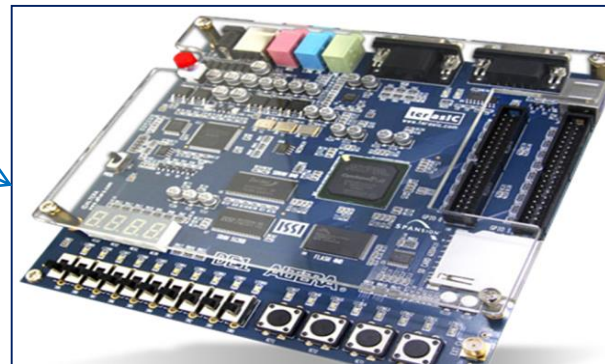
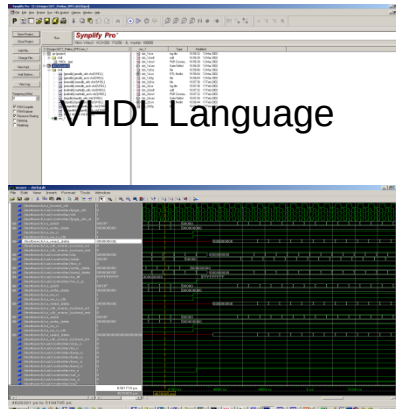
```
entity INVERSEUR is  
Port (A: in std_logic;  
      S: out std_logic);  
end INVERSEUR ;
```

```
architecture DESCRIPTION of INVERSEUR  
is  
begin  
S <= not(A);  
end DESCRIPTION;
```



INVERSEUR

Déclaration de l'architecture du décodeur
Correspondance schématique

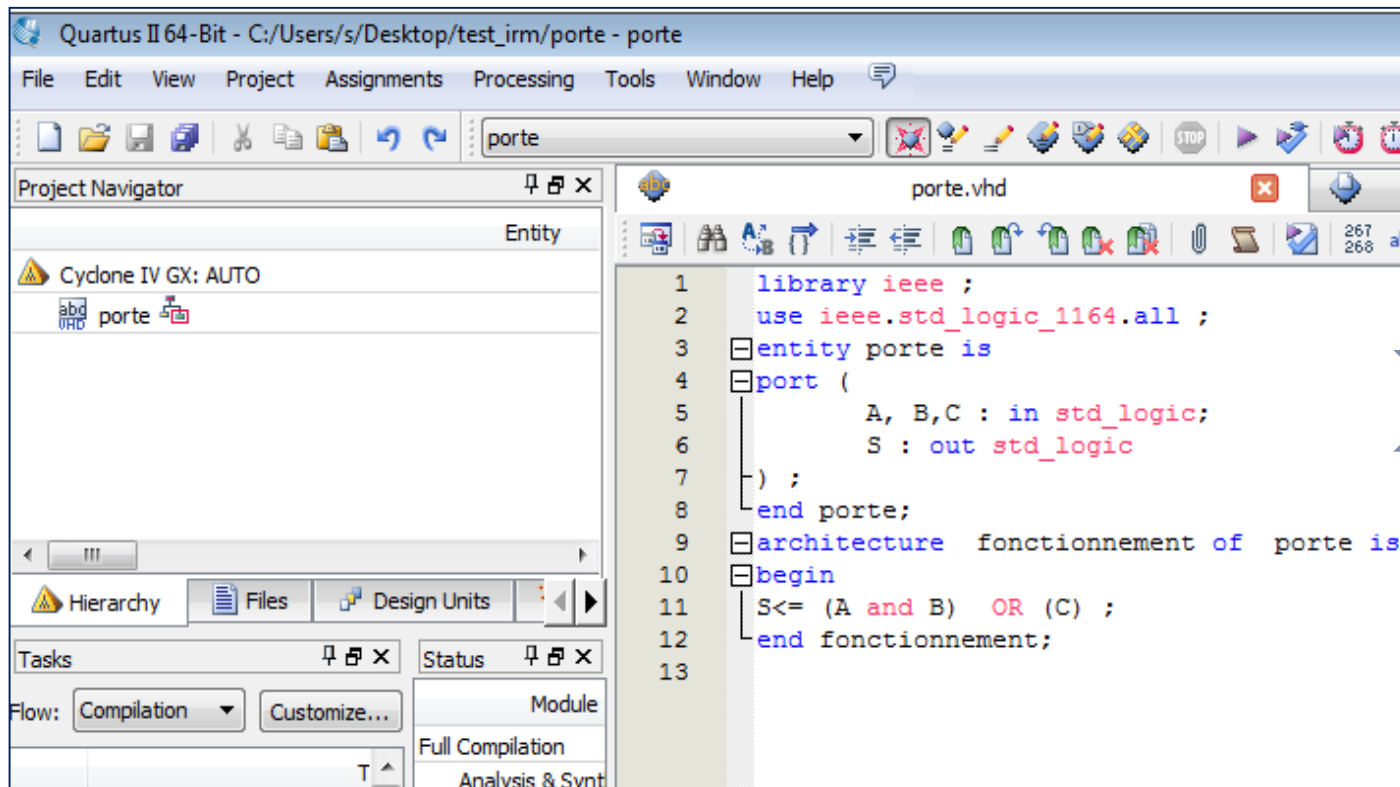


Carte FPGA Altera Cyclone II

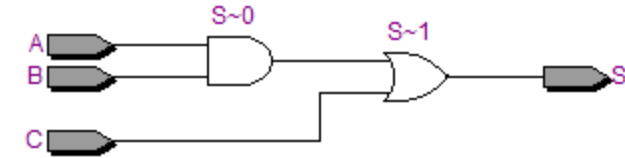
III. Conception matérielle

1. Cible Matériel
2. Technologie de base: Les transistors
3. Evolution des systèmes embarqués
4. Exemples de Systèmes Embarqués

VHDL



library ieee ;
use ieee.std_logic_1164.all ;
entity porte is
port (
 A, B,C : in std_logic;
 S : out std_logic
);
end porte;
architecture fonctionnement of porte is
begin
S<= (A or B) and (A OR (not (C))) ;
end fonctionnement;



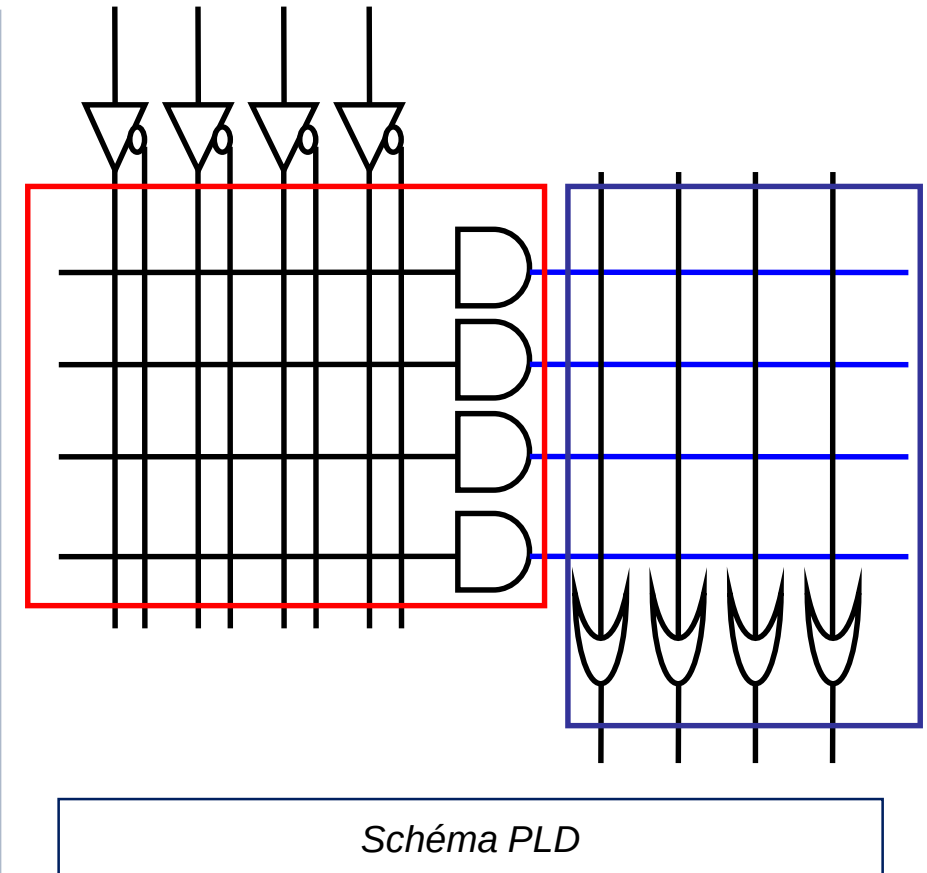
Les PLDs: Programmable Logic Devices

- ❑ Matrice de « ET » programmables réalisant tous les produits

possibles connectée aux sorties par des « OU » programmables

- ❑ Grande surface de Silicium utilisée

- ❑ Ces circuits ne sont plus utilisés aujourd'hui



Les PLDs: Programmable Logic Devices

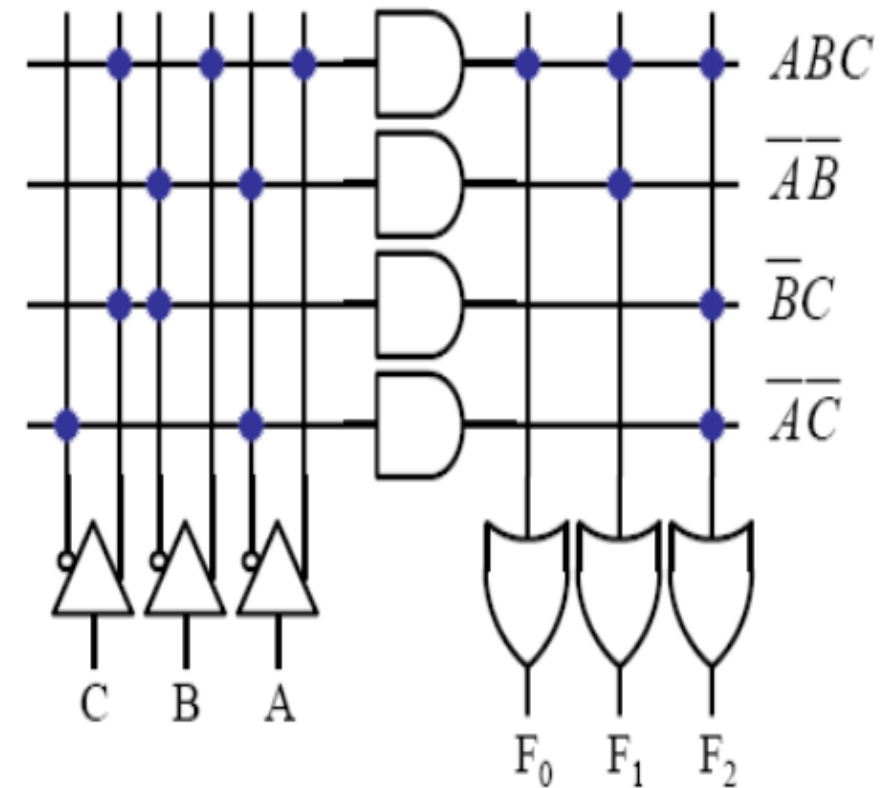
❑ Exemple

❑ La configuration du circuit assurant les fonctions suivantes:

❑ $F_0 = ABC$

❑ $F_1 = ABC + \bar{A}\bar{B}$

❑ $F_2 = ABC + \bar{B}C + \bar{A}\bar{C}$



Plan du cours

Définition et généralités

Conception des systèmes embarqués

Conception matérielle

Conception logicielle

Conception Mixte

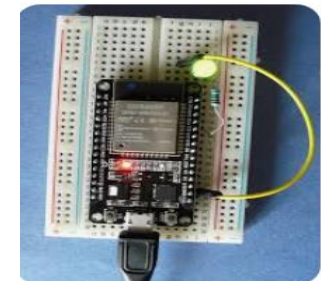
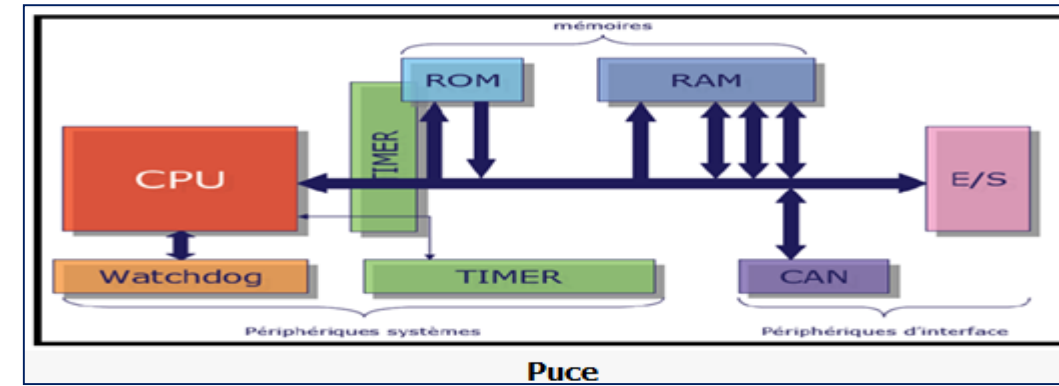
Résumé

Les microcontrôleurs

❑ Ce sont des cibles programmables, c'est-à-dire qu'on peut modifier l'application dédiée juste en modifiant le code, à travers :

❑ Un microcontrôleur est un système électronique et informatique autonome qui regroupe un processeur, de la mémoire et des périphériques sur une seule puce, conçu pour des tâches spécifiques comme dans les systèmes embarqués.

❑ Contrairement au microprocesseur, qui ne contient que le CPU et nécessite des composants externes, le microcontrôleur est une solution complète, souvent utilisée dans les systèmes embarqués.

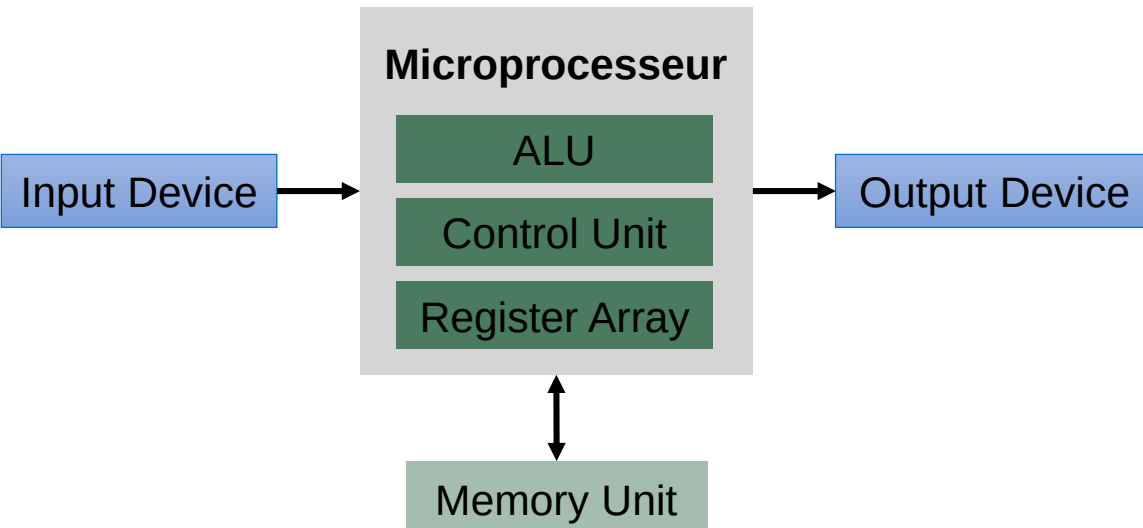


Microcontrôleurs (MCU) vs. Microprocesseurs (MPU)

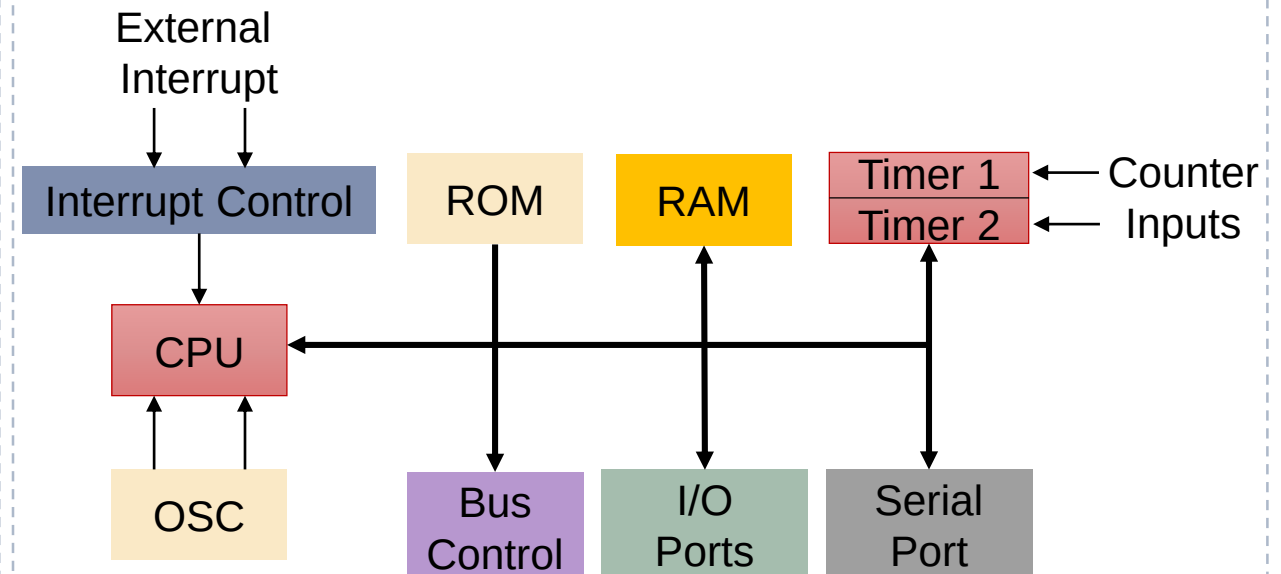
Critère	Microprocesseur (MPU)	Microcontrôleur (MCU)
Architecture	CPU seul, mémoire externe	CPU + mémoire + périphériques intégrés
Objectif	Calcul intensif, multitâche	Contrôle dédié, temps réel
Consommation	Élevée	Faible
Complexité logicielle	OS complet (Linux, Windows)	Firmware léger, souvent bare-metal
Applications	PC, serveurs, smartphones	IoT, capteurs, systèmes embarqués
Exemples	Intel i7, ARM Cortex-A	STM32, PIC, Arduino

Microcontrôleurs (MCU) vs. Microprocesseurs (MPU)

Microprocesseur: Nécessite des composants externes (RAM, ROM, périphériques).



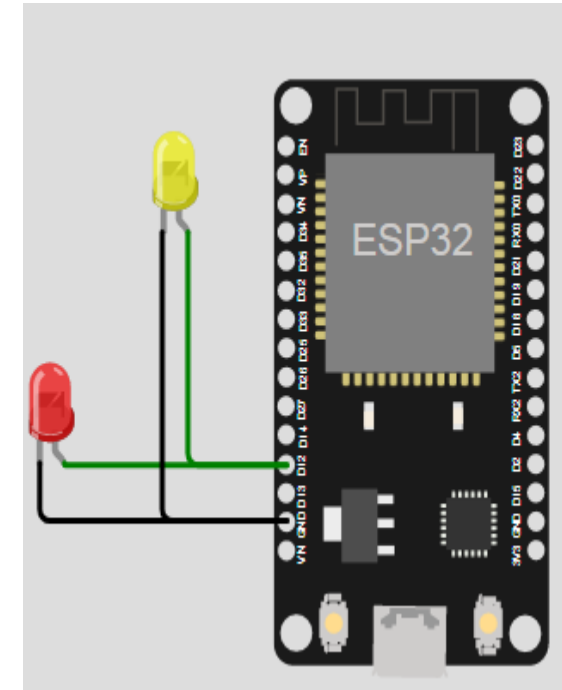
Microcontrôleurs: Intègre CPU, mémoire (RAM/ROM/Flash), timers, convertisseurs A/N, interfaces sur même puce



1. Les microcontrôleurs
2. **Approche Logicielle**
3. Cible Logiciel vs cible Matériel

Approche Logicielle

```
sketch.ino • diagram.json Library Manager ▼  
1 // Déclaration de la broche GPIO à laquelle la LED est connectée  
2 const int ledPin = 12;  
3  
4 void setup() {  
5   // Configuration de la broche LED en tant que sortie  
6   pinMode(ledPin, OUTPUT);  
7 }  
8  
9 void loop() {  
10  // Allumer la LED  
11  digitalWrite(ledPin, HIGH);  
12  // Attendre pendant 1 seconde  
13  delay(1000);  
14  // Éteindre la LED  
15  digitalWrite(ledPin, LOW);  
16  // Attendre pendant 1 seconde  
17  delay(1000); }  
18
```



Approche Logicielle

❑ Table de vérité:

❑ Voici la table de vérité pour le contrôle de la LED en fonction des états des boutons A, B et C:

A	B	C	LED
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1



Nous souhaitons réaliser un système embarqué permettant le contrôle d'une LED via trois boutons-poussoirs. Ce système sera conçu pour répondre à des conditions spécifiques d'interaction, où l'état des boutons déterminera l'allumage ou l'extinction de la LED.

Approche Logicielle

```
// Define GPIO pins for the three push buttons
const int buttonPinA = 7; // Button A connected
to GPIO 7
const int buttonPinB = 5; // Button B connected
to GPIO 5
const int buttonPinC = 4; // Button C connected
to GPIO 4
```

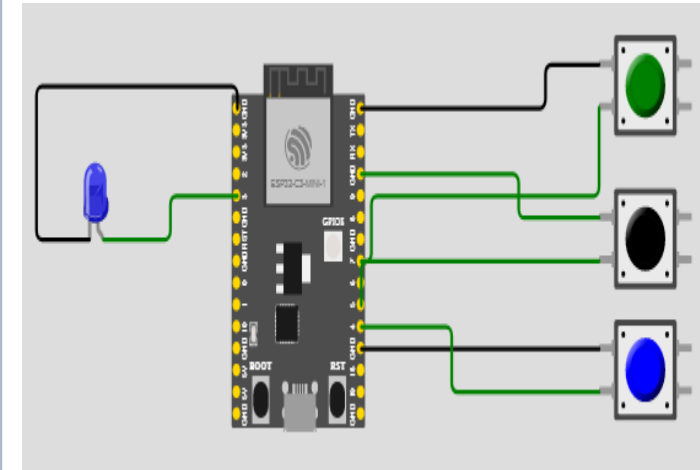
```
// Define GPIO pin for LED
const int ledPin = 3; // LED connected to GPIO
3
```

```
void setup() {
  // Initialize the LED pin as an output
  pinMode(ledPin, OUTPUT);
}
```

```
// Initialize the button pins as inputs with pull-up resistors
pinMode(buttonPinA, INPUT_PULLUP);
pinMode(buttonPinB, INPUT_PULLUP);
pinMode(buttonPinC, INPUT_PULLUP);
}
```

```
void loop() {
  // Read the state of each button (LOW means pressed)
  bool stateA = (digitalRead(buttonPinA) == LOW);
  bool stateB = (digitalRead(buttonPinB) == LOW);
  bool stateC = (digitalRead(buttonPinC) == LOW);
```

```
// Check the condition: s = a AND b OR c
if ((stateA || stateB) || stateC) {
  digitalWrite(ledPin, HIGH); // Turn on the LED
} else {
  digitalWrite(ledPin, LOW); // Turn off the LED
}
}
```



Approche Logicielle



Avantages

- **Flexibilité:** il suffit de modifier le programme pour modifier l'application
- **Simple à mettre en œuvre:** grâce à la programmation de haut niveau (langage C) (possibilité de grande abstraction par rapport au matériel)
- **Temps et coût de conception faibles**
- **Prix de reviens faible**



Inconvénients

- **Faibles performance:** temps d'exécution élevé
- **Consommation électrique importante**
- **Le passage par un compilateur peut dégrader les performances**
- **Complexité du langage machine et de l'assembleur**

Cible Logiciel vs cible Matériel

	Logiciel	Matériel
Avantages	✓ Réduction de la surface ✓ Partie contrôle prépondérante ✓ Connexions avec d'autres modules ✓ Fonctions spécialisées disponibles dans l'UAL du processeur ✓ Evolution / Flexibilité ✓ Coût	✓ Meilleure performance ✓ Traitement du parallélisme ✓ Traitement de données ✓ Interactions avec l'extérieur ✓ Connexions a d'autres modules matériels
Inconvénients	✗ Lenteur relative ✗ Communication avec le matériel ✗ Synchronisation avec le matériel	✗ Coût ✗ Surface ✗ Communication avec le logiciel ✗ Synchronisation avec le logiciel

Plan du cours

Définition et généralités

Conception des systèmes embarqués

Conception matérielle

Conception logicielle

Conception Mixte

Résumé

Conception mixte

i La conception mixte logicielle et matérielle pour les systèmes embarqués consiste à répartir intelligemment les fonctions d'un système entre le logiciel (programmable, flexible) et le matériel (rapide, spécialisé), afin d'obtenir un compromis optimal entre performance, coût, consommation et délai de développement.

Principes de la conception mixte

Partitionnement logiciel/matériel : déterminer quelles parties de l'application doivent être exécutées en logiciel (processeur, microcontrôleur) et lesquelles doivent être implémentées en matériel (FPGA, ASIC, DSP).

Codesign : approche intégrée où matériel et logiciel sont conçus simultanément, avec des outils permettant de simuler et valider l'ensemble.

Objectif : optimiser la performance tout en gardant une flexibilité suffisante pour les mises à jour et évolutions

Conception mixte

⚙️ Étapes clés

Analyse des besoins:

- Identifier contraintes de temps réel, consommation énergétique, coût, taille mémoire.

Modélisation:

- Utiliser UML, SystemC ou Matlab/Simulink pour décrire le système global.

Partitionnement:

- Fonctions critiques → matériel (accélération, parallélisme).
- Fonctions évolutives ou complexes → logiciel (programmation plus simple).

Implémentation:

- Logiciel: C/C++, assembleur, OS embarqué (Linux embarqué, FreeRTOS).

Matériel:

- HDL (VHDL, Verilog), FPGA ou ASIC.

Validation et co-simulation:

- Tester l'interaction logiciel/matériel avant prototypage réel.

Exemples d'applications

Multimédia : encodeurs/décodeurs vidéo où le traitement intensif est en matériel, le contrôle en logiciel.

Télécommunications : protocoles gérés en logiciel, traitement du signal en FPGA/DSP.

Automobile : logiciels de contrôle (ABS, injection) associés à des circuits spécialisés pour la rapidité.

IoT : microcontrôleurs exécutant le logiciel, avec accélérateurs matériels pour cryptographie ou radio.

Conception mixte

⚠ Points critiques et défis

- ❑ Complexité croissante : les systèmes embarqués modernes (multimédia, télécoms, IA) exigent une intégration fine.
- ❑ Choix d'architecture : compromis entre flexibilité, performance, consommation et coût.
- ❑ Outils de conception : nécessité de maîtriser à la fois les langages logiciels (C, C++) et matériels (VHDL, Verilog).
- ❑ Validation : la co-simulation est indispensable pour éviter des erreurs coûteuses en production.

Plan du cours

Définition et généralités

Conception des systèmes embarqués

Conception matérielle

Conception logicielle

Conception Mixte

Résumé

Résumé

- ❑ Système Embarqué “Embedded system”: tout système conçu pour résoudre un problème ou une tâche spécifique mais n'est pas un ordinateur d'usage général.
- ❑ Utilisent généralement un microprocesseur combiné avec d'autres matériel et logiciel
- ❑ Système électronique et informatique autonome ne possédant pas des entrées-sorties standards.
- ❑ **La conception mixte logicielle/matérielle est au cœur des systèmes embarqués modernes** : elle permet de tirer parti de la flexibilité du logiciel et de la puissance du matériel. Le défi est de trouver le bon équilibre selon les contraintes du projet.